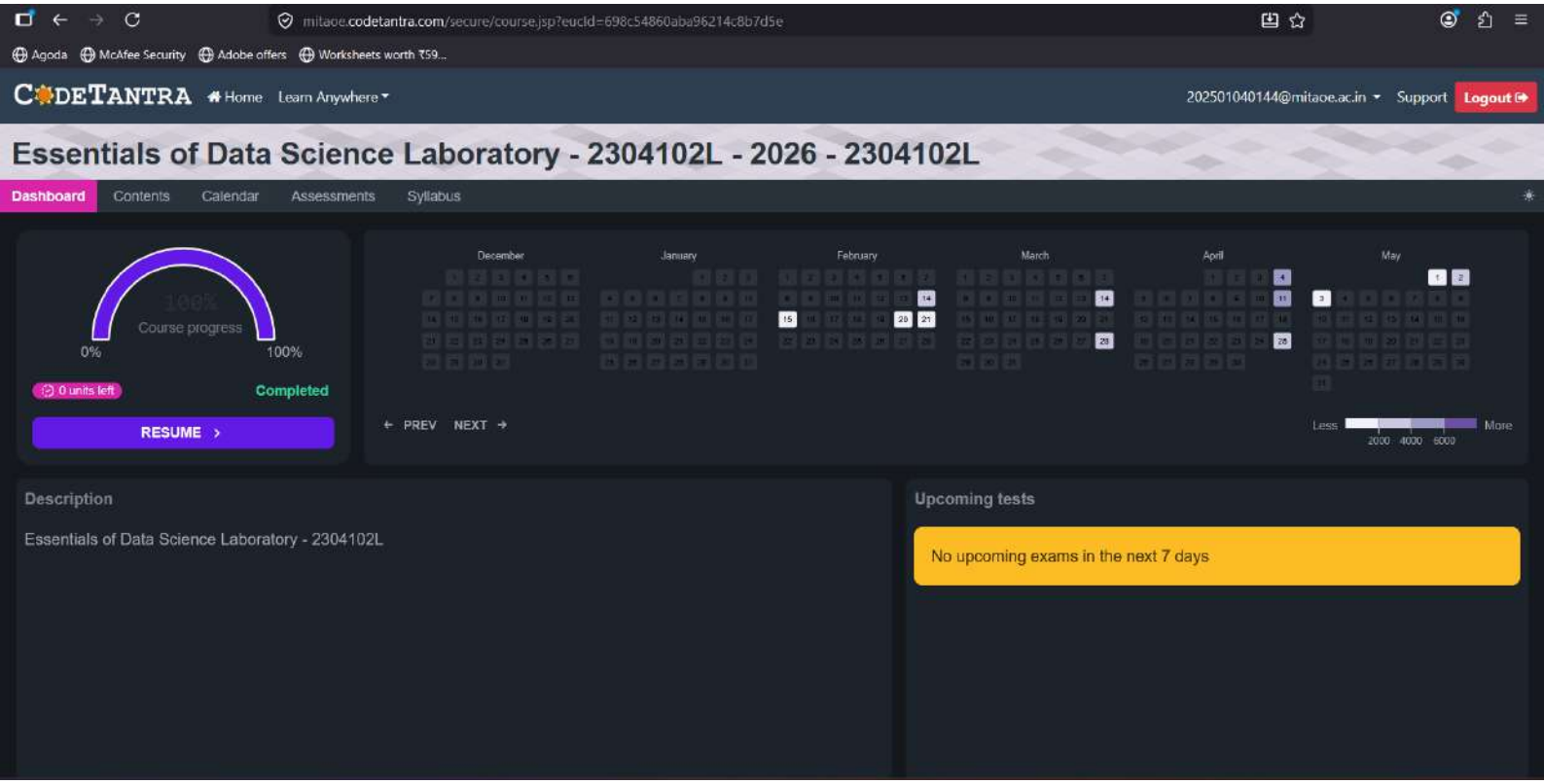
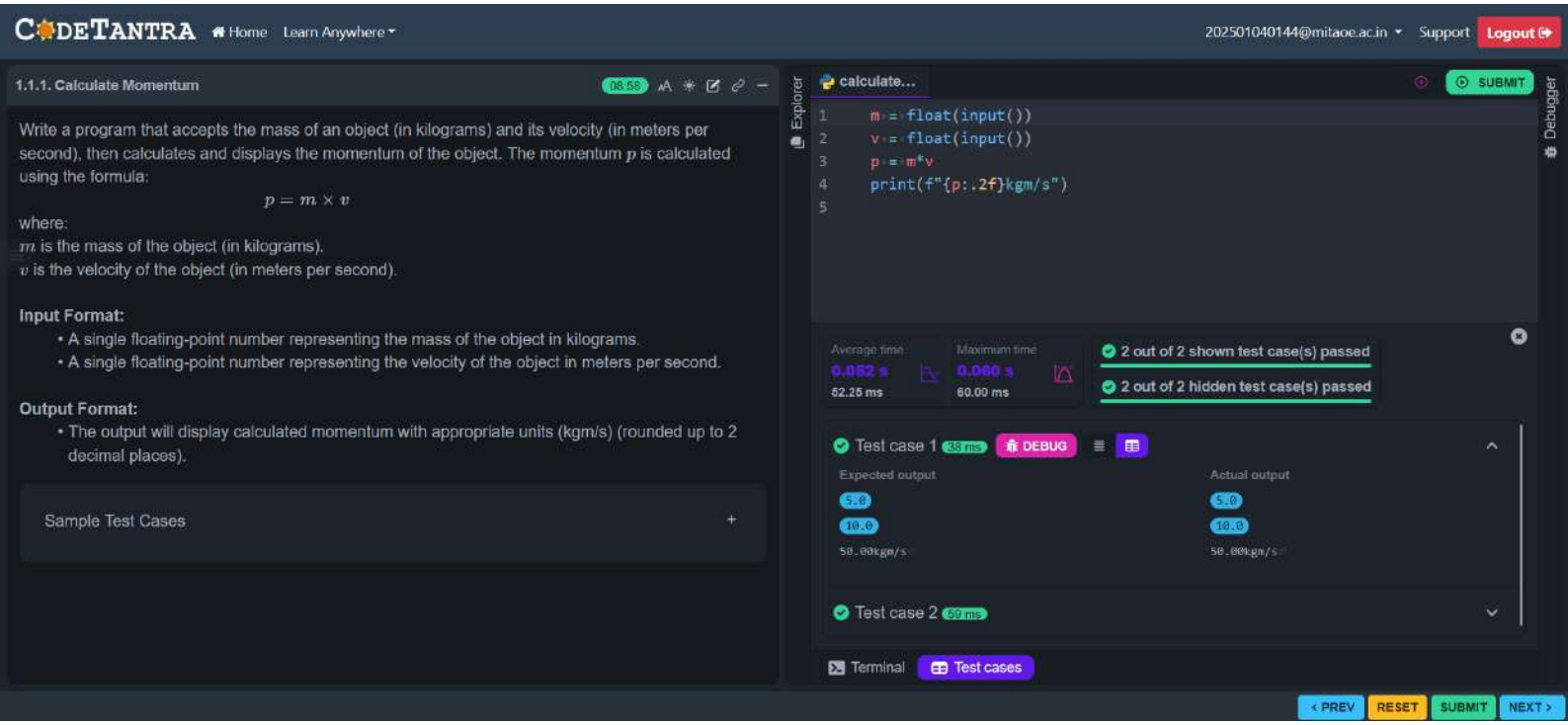
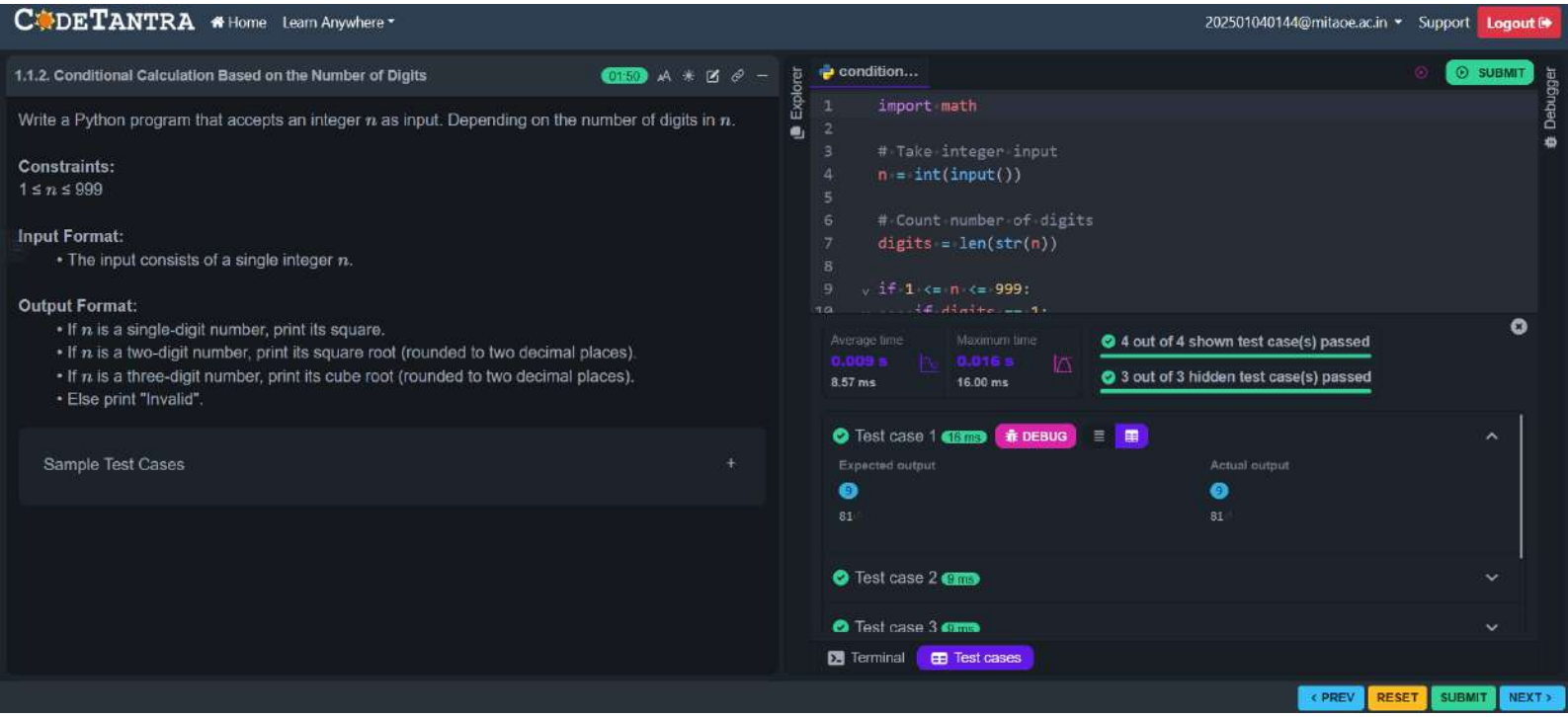
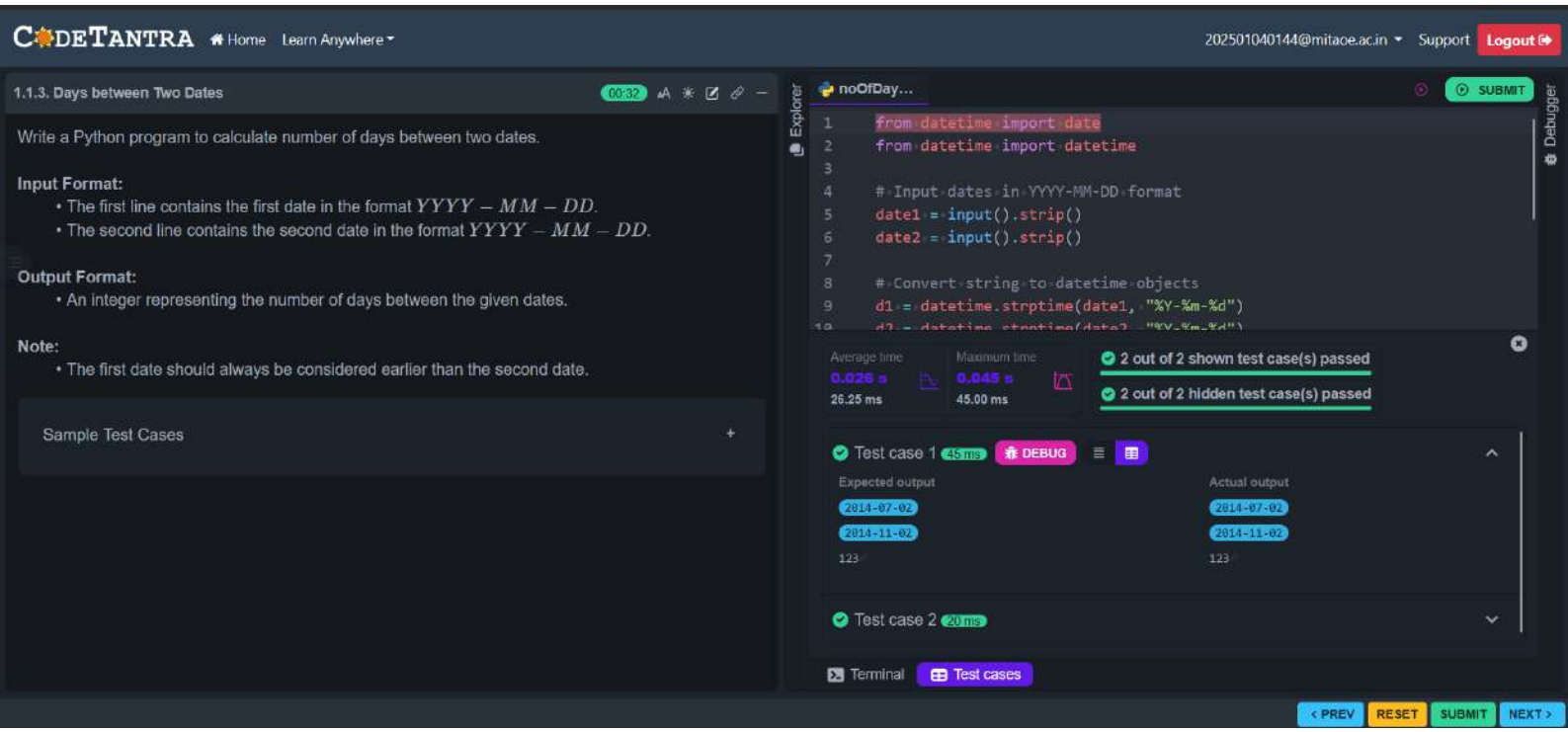
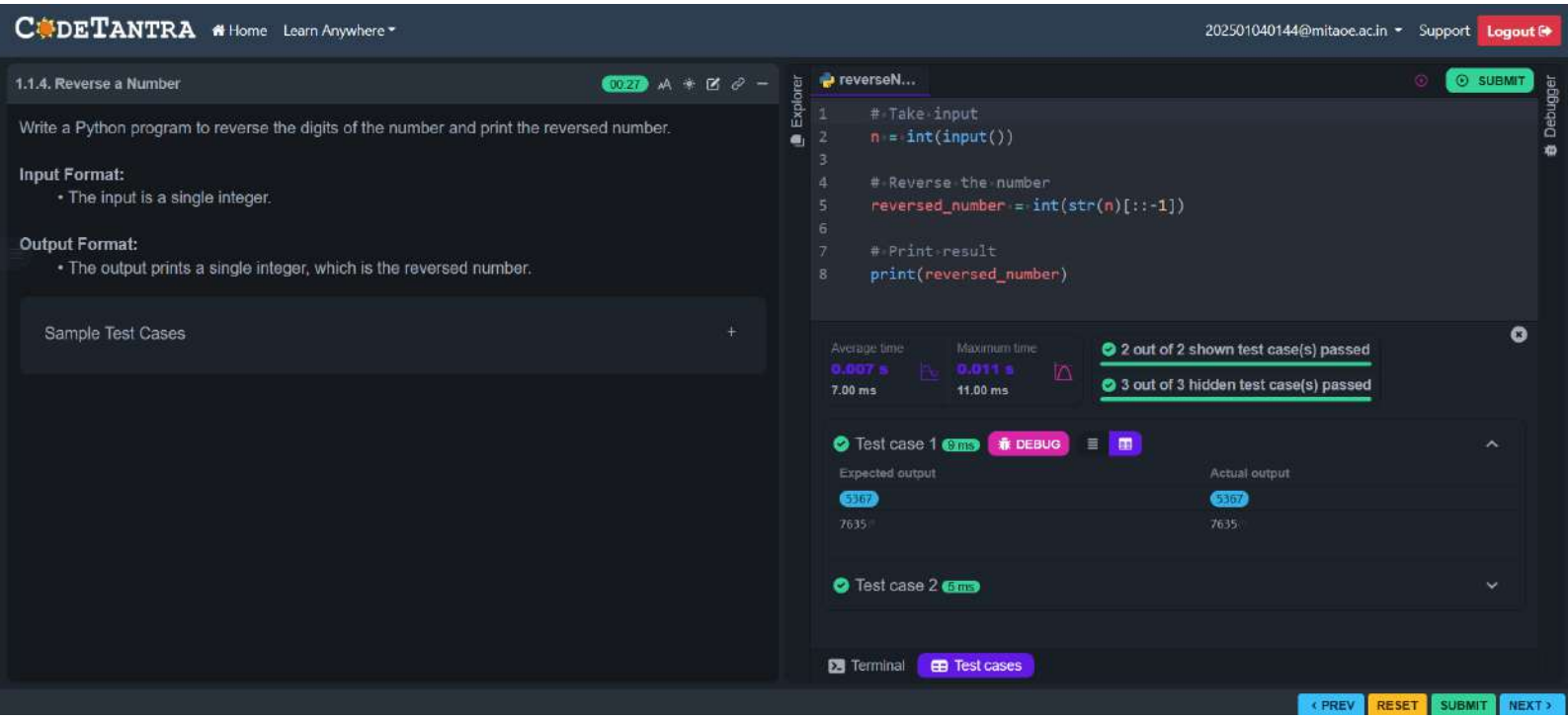
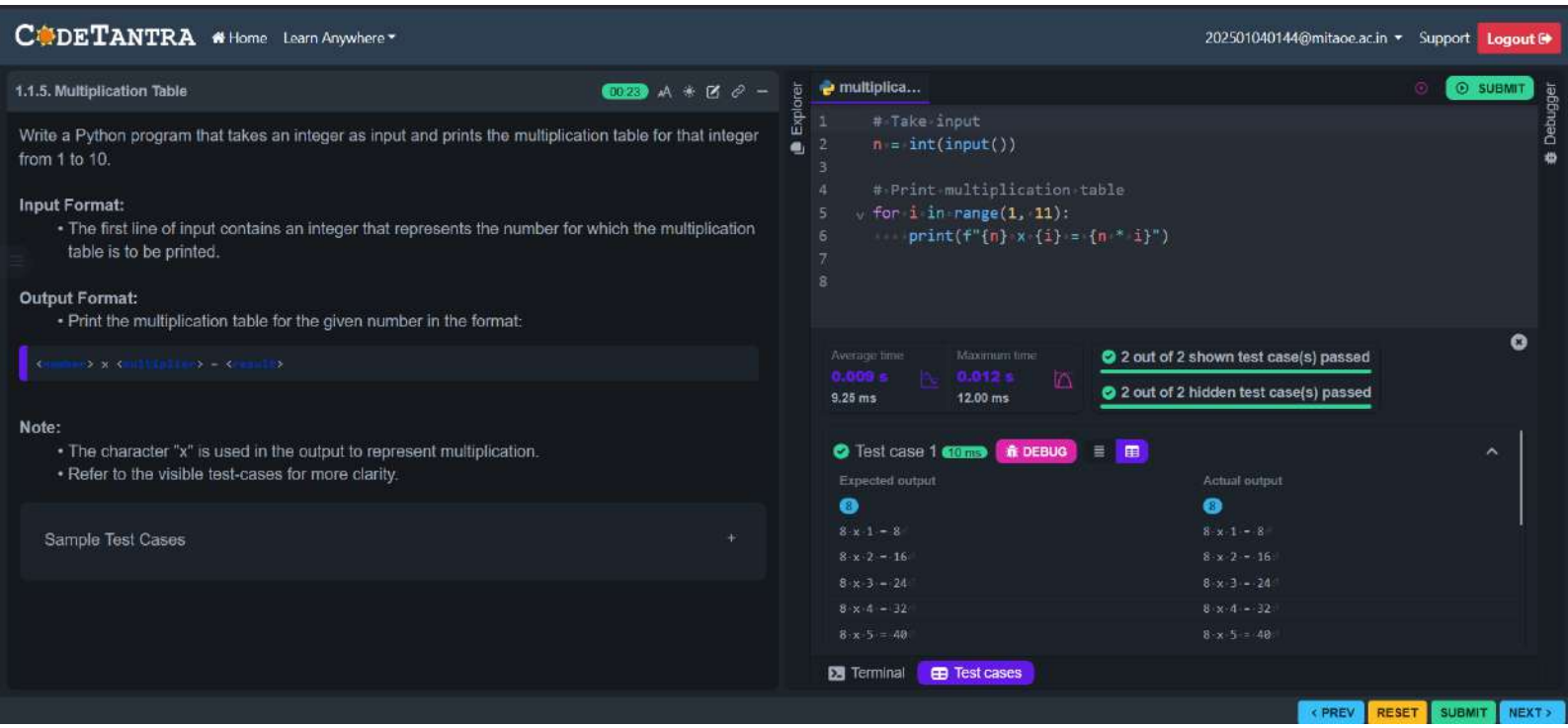

mitaoe.codetantra.com/secure/course.jsp?eucId=698c54860aba96214c8b7d5e
Agoda
McAfee Security
Adobe offers
Worksheets worth ₹59...
CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
Essentials of Data Science Laboratory - 2304102L - 2026 - 2304102L
Dashboard
Contents
Calendar
Assessments
Syllabus
100%
Course progress
0%
100%
0 units left
Completed
RESUME
December
January
February
March
April
May
PREV
NEXT
Less
More
2000
4000
6000
Description
Essentials of Data Science Laboratory - 2304102L
Upcoming tests
No upcoming exams in the next 7 days

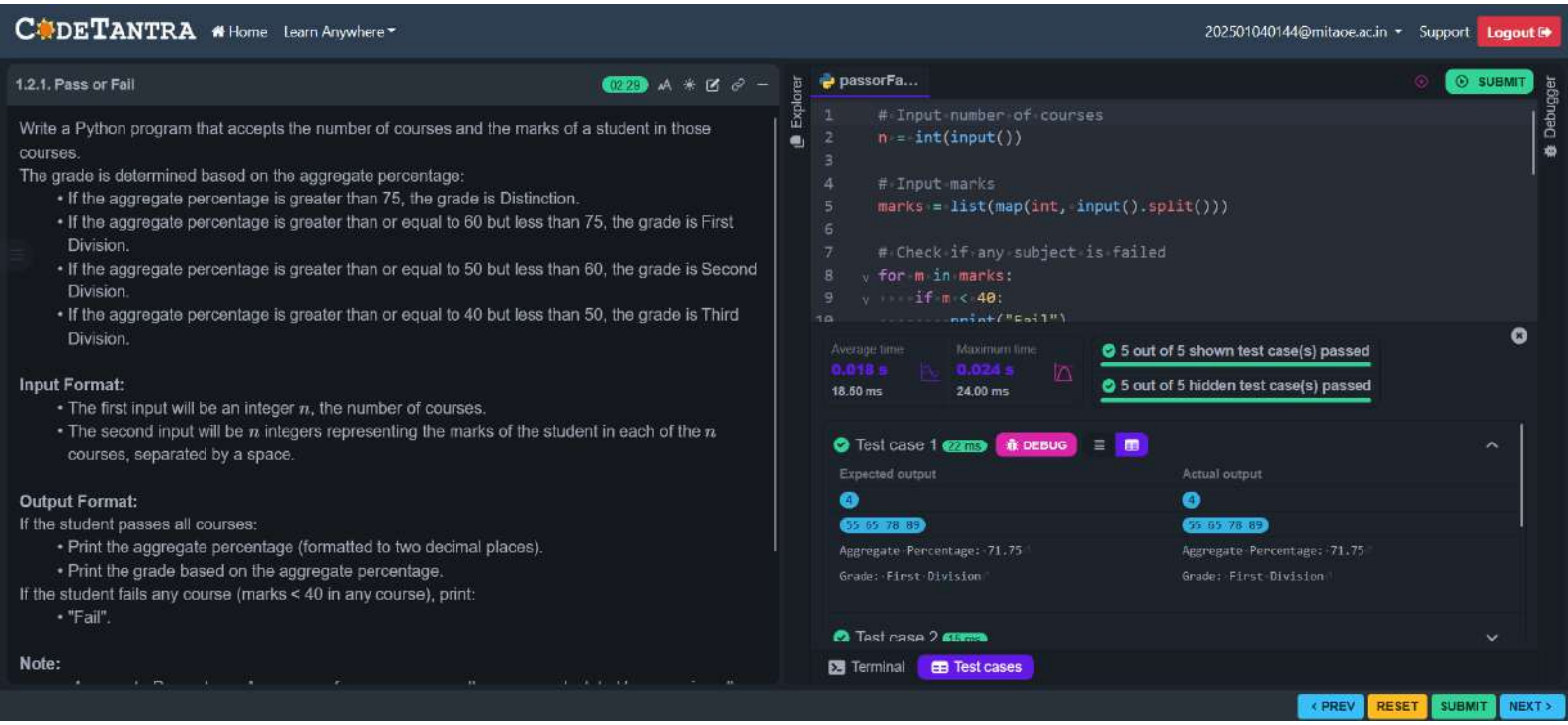

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
1.1.1. Calculate Momentum
08:58
Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:
$$p = m \times v$$
where:
m is the mass of the object (in kilograms).
v is the velocity of the object (in meters per second).
Input Format:
• A single floating-point number representing the mass of the object in kilograms.
• A single floating-point number representing the velocity of the object in meters per second.
Output Format:
• The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).
Sample Test Cases
Explorer
calculate...
SUBMIT
Debugger
1 m = float(input())
2 v = float(input())
3 p = m*v
4 print(f"{p:.2f}kgm/s")
5
Average time
0.052 s
52.25 ms
Maximum time
0.060 s
60.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
38 ms
DEBUG
Expected output
5.0
10.0
50.00kgm/s
Actual output
5.0
10.0
50.00kgm/s
Test case 2
59 ms
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

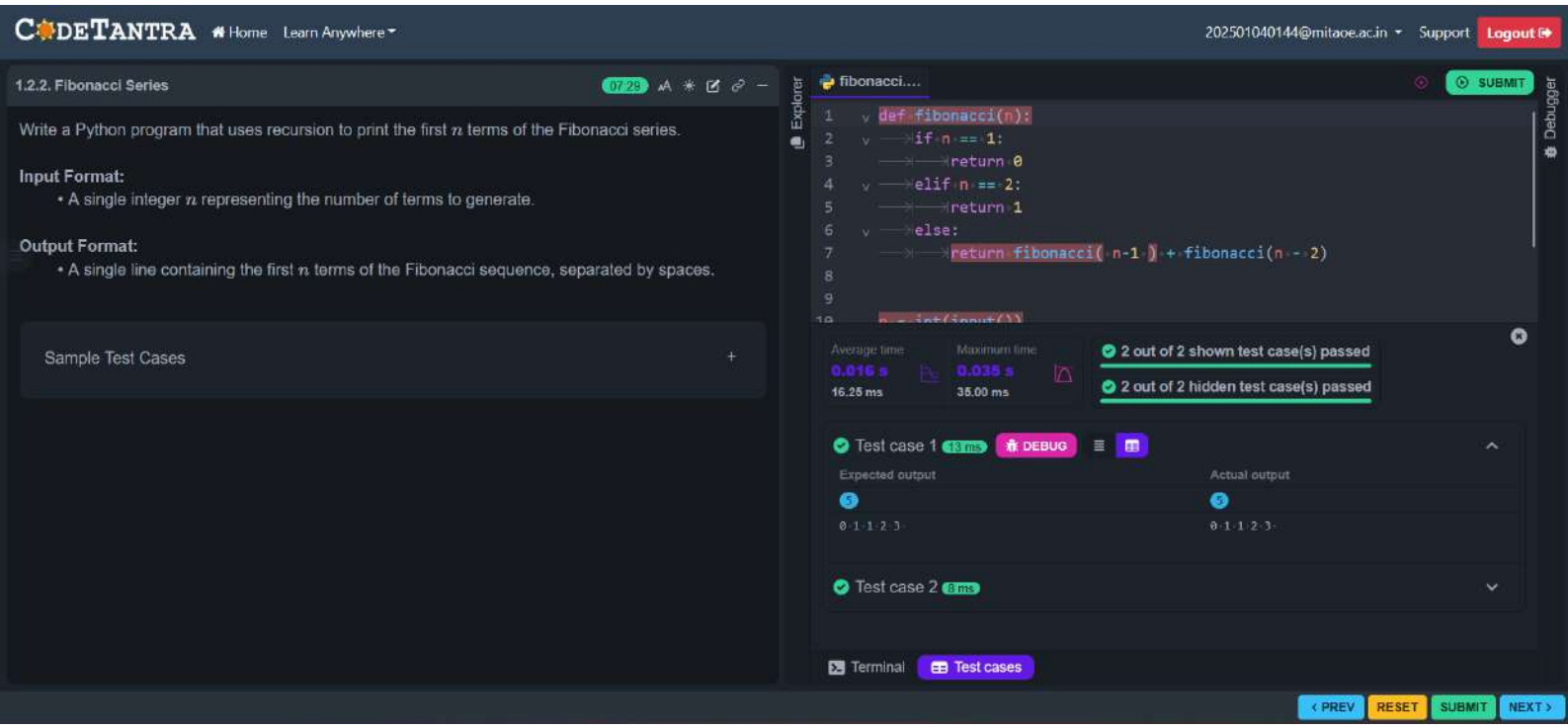

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
1.1.2. Conditional Calculation Based on the Number of Digits
01:50
Write a Python program that accepts an integer n as input. Depending on the number of digits in n.
Constraints:
$1 \leq n \leq 999$
Input Format:
• The input consists of a single integer n.
Output Format:
• If n is a single-digit number, print its square.
• If n is a two-digit number, print its square root (rounded to two decimal places).
• If n is a three-digit number, print its cube root (rounded to two decimal places).
• Else print "Invalid".
Sample Test Cases
Explorer
condition...
SUBMIT
Debugger
1 import math
2
3 # Take integer input
4 n = int(input())
5
6 # Count number of digits
7 digits = len(str(n))
8
9 if 1 <= n <= 999:
Average time
0.009 s
8.57 ms
Maximum time
0.016 s
16.00 ms
4 out of 4 shown test case(s) passed
3 out of 3 hidden test case(s) passed
Test case 1
16 ms
DEBUG
Expected output
9
81
Actual output
9
81
Test case 2
9 ms
Test case 3
0 ms
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

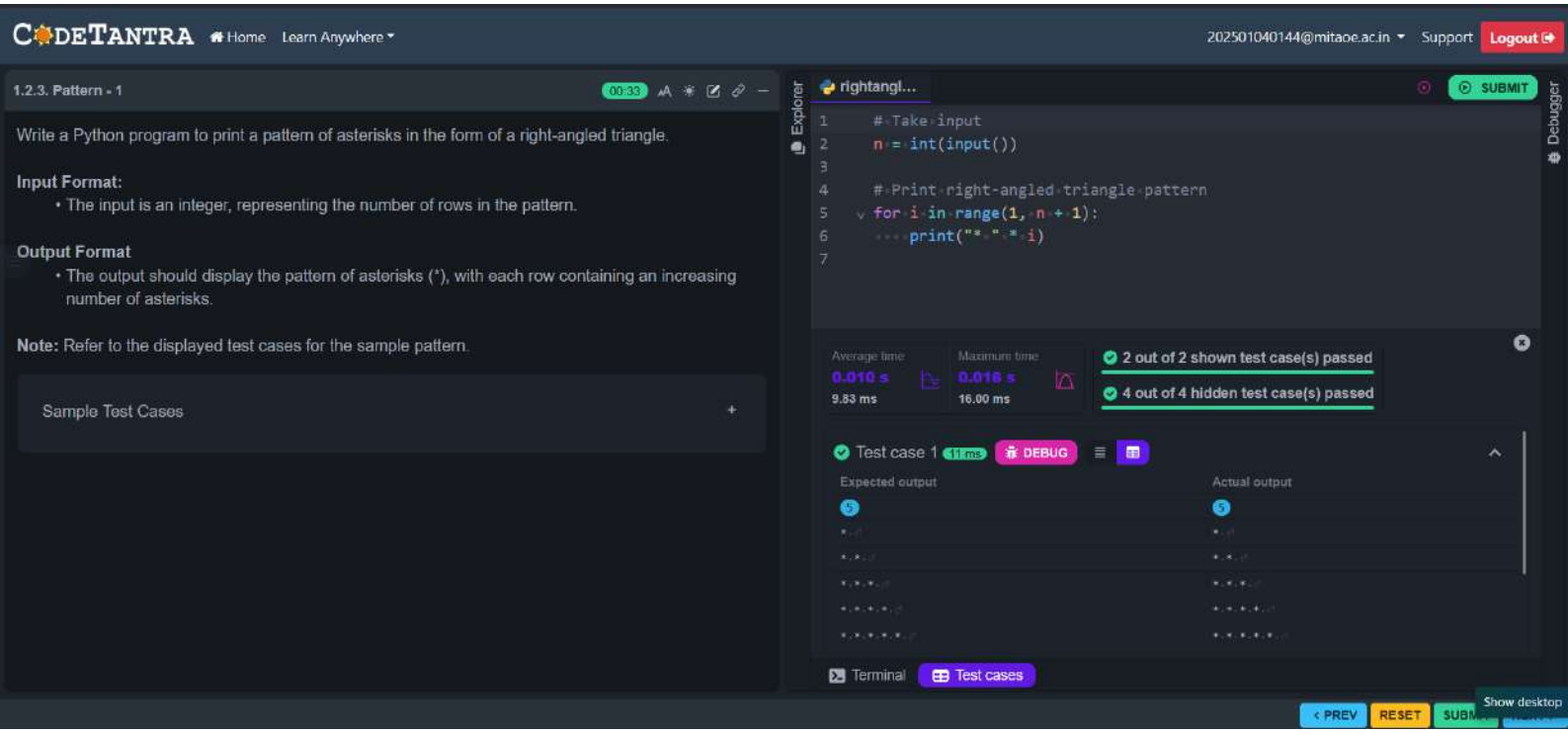

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
1.1.3. Days between Two Dates
00:32
Write a Python program to calculate number of days between two dates.
Input Format:
• The first line contains the first date in the format YYYY − MM − DD.
• The second line contains the second date in the format YYYY − MM − DD.
Output Format:
• An integer representing the number of days between the given dates.
Note:
• The first date should always be considered earlier than the second date.
Sample Test Cases
noOfDay...
SUBMIT
Explorer
Debugger
from datetime import date
from datetime import datetime
# Input dates in YYYY-MM-DD format
date1 = input().strip()
date2 = input().strip()
# Convert string to datetime objects
d1 = datetime.strptime(date1, "%Y-%m-%d")
Average time
0.026 s
26.25 ms
Maximum time
0.045 s
45.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1 45 ms
DEBUG
Expected output
2014-07-02
2014-11-02
123
Actual output
2014-07-02
2014-11-02
123
Test case 2 20 ms
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

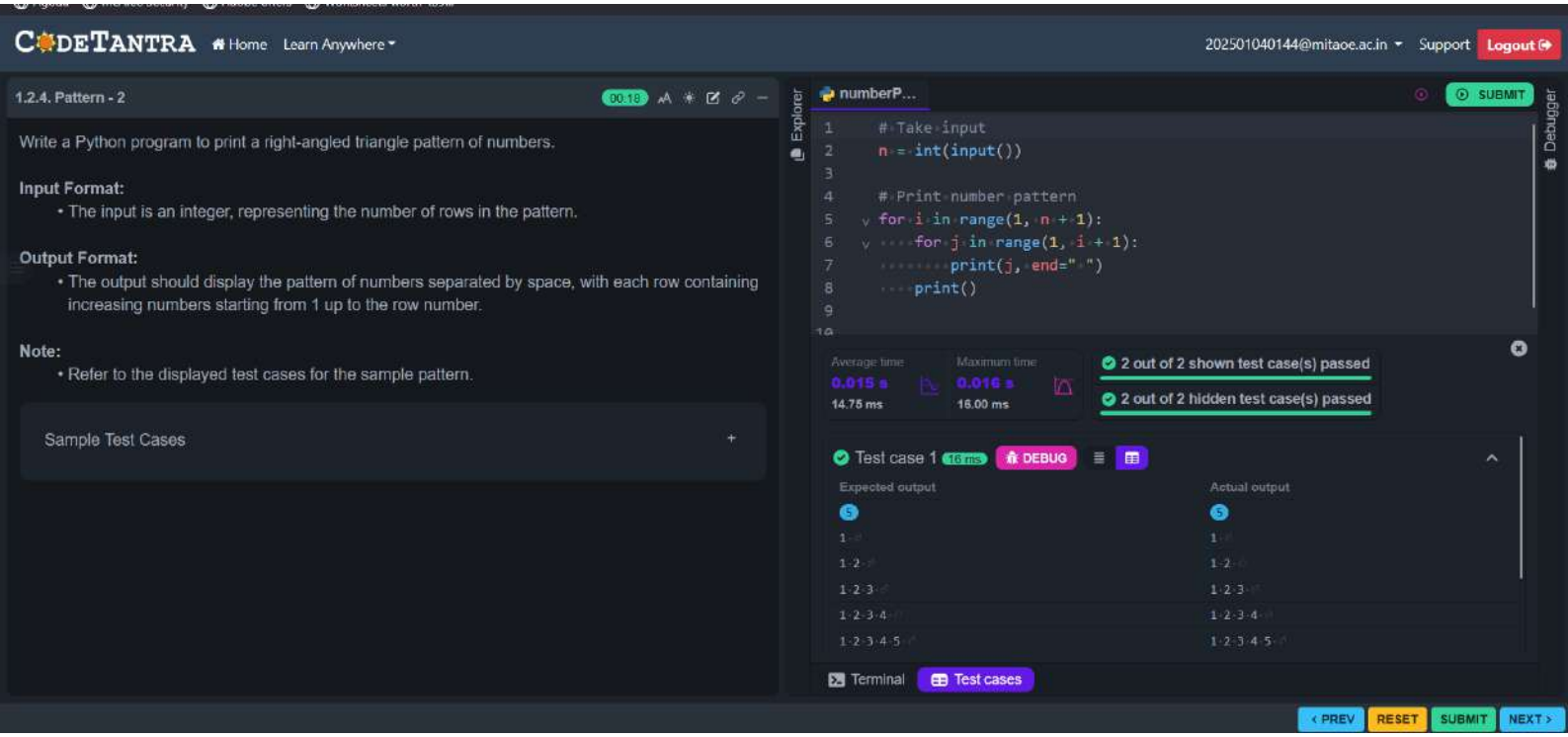

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
1.1.4. Reverse a Number
00:27
Write a Python program to reverse the digits of the number and print the reversed number.
Input Format:
• The input is a single integer.
Output Format:
• The output prints a single integer, which is the reversed number.
Sample Test Cases
Explorer
reverseN...
SUBMIT
Debugger
# Take input
n = int(input())
# Reverse the number
reversed_number = int(str(n)[::-1])
# Print result
print(reversed_number)
Average time
0.007 s
7.00 ms
Maximum time
0.011 s
11.00 ms
2 out of 2 shown test case(s) passed
3 out of 3 hidden test case(s) passed
Test case 1
9 ms
DEBUG
Expected output
5367
7635
Actual output
5367
7635
Test case 2
6 ms
Terminal
Test cases
PREV
RESET
SUBMIT
NEXT

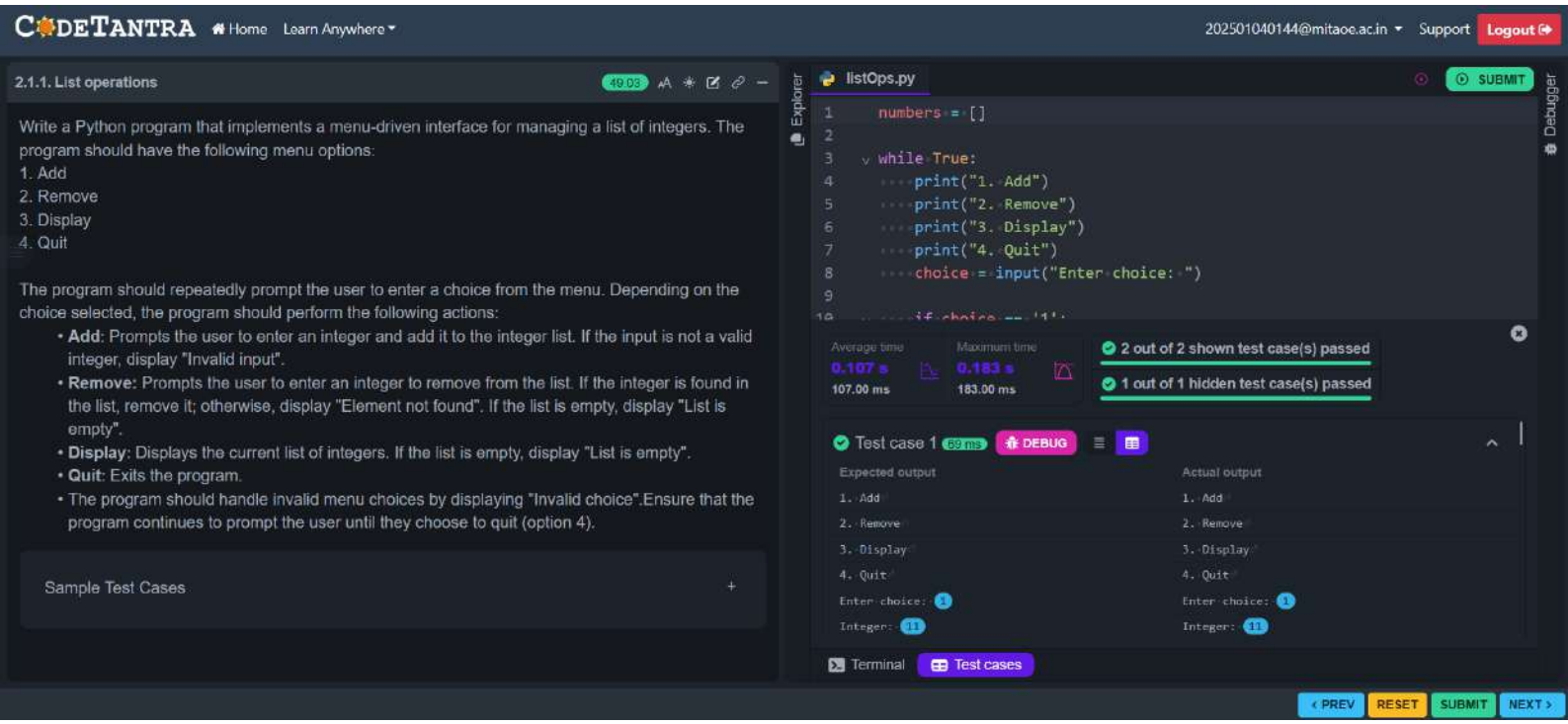

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
1.1.5. Multiplication Table
00:23
Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.
Input Format:
• The first line of input contains an integer that represents the number for which the multiplication table is to be printed.
Output Format:
• Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>
Note:
• The character "x" is used in the output to represent multiplication.
• Refer to the visible test-cases for more clarity.
Sample Test Cases
Explorer
multiplica...
SUBMIT
Debugger
# Take input
n = int(input())
# Print multiplication table
for i in range(1, 11):
print(f"{n} x {i} = {n * i}")
Average time
0.009 s
9.25 ms
Maximum time
0.012 s
12.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
10 ms
DEBUG
Expected output
8
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
Actual output
8
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

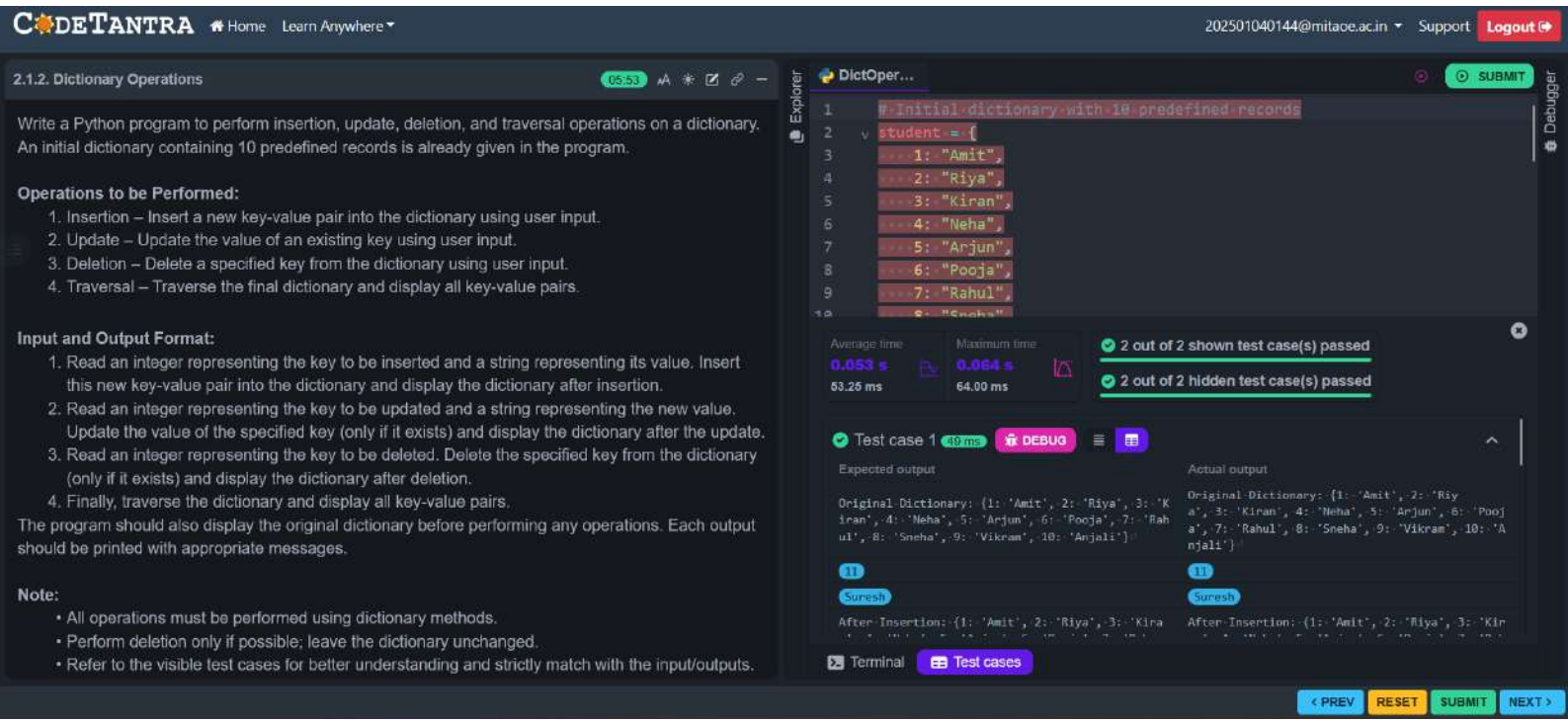

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
1.2.1. Pass or Fail
02:28
Write a Python program that accepts the number of courses and the marks of a student in those courses.
The grade is determined based on the aggregate percentage:
• If the aggregate percentage is greater than 75, the grade is Distinction.
• If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
• If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
• If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.
Input Format:
• The first input will be an integer n, the number of courses.
• The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.
Output Format:
If the student passes all courses:
• Print the aggregate percentage (formatted to two decimal places).
• Print the grade based on the aggregate percentage.
If the student fails any course (marks < 40 in any course), print:
• "Fail".
Note:
Explorer
passorFa...
SUBMIT
Debugger
# Input number of courses
n = int(input())
# Input marks
marks = list(map(int, input().split()))
# Check if any subject is failed
for m in marks:
if m < 40:
print("Fail")
Average time
0.018 s
18.50 ms
Maximum time
0.024 s
24.00 ms
5 out of 5 shown test case(s) passed
5 out of 5 hidden test case(s) passed
Test case 1
22 ms
DEBUG
Expected output
4
55 65 78 89
Aggregate Percentage: 71.75
Grade: First Division
Actual output
4
55 65 78 89
Aggregate Percentage: 71.75
Grade: First Division
Test case 2
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

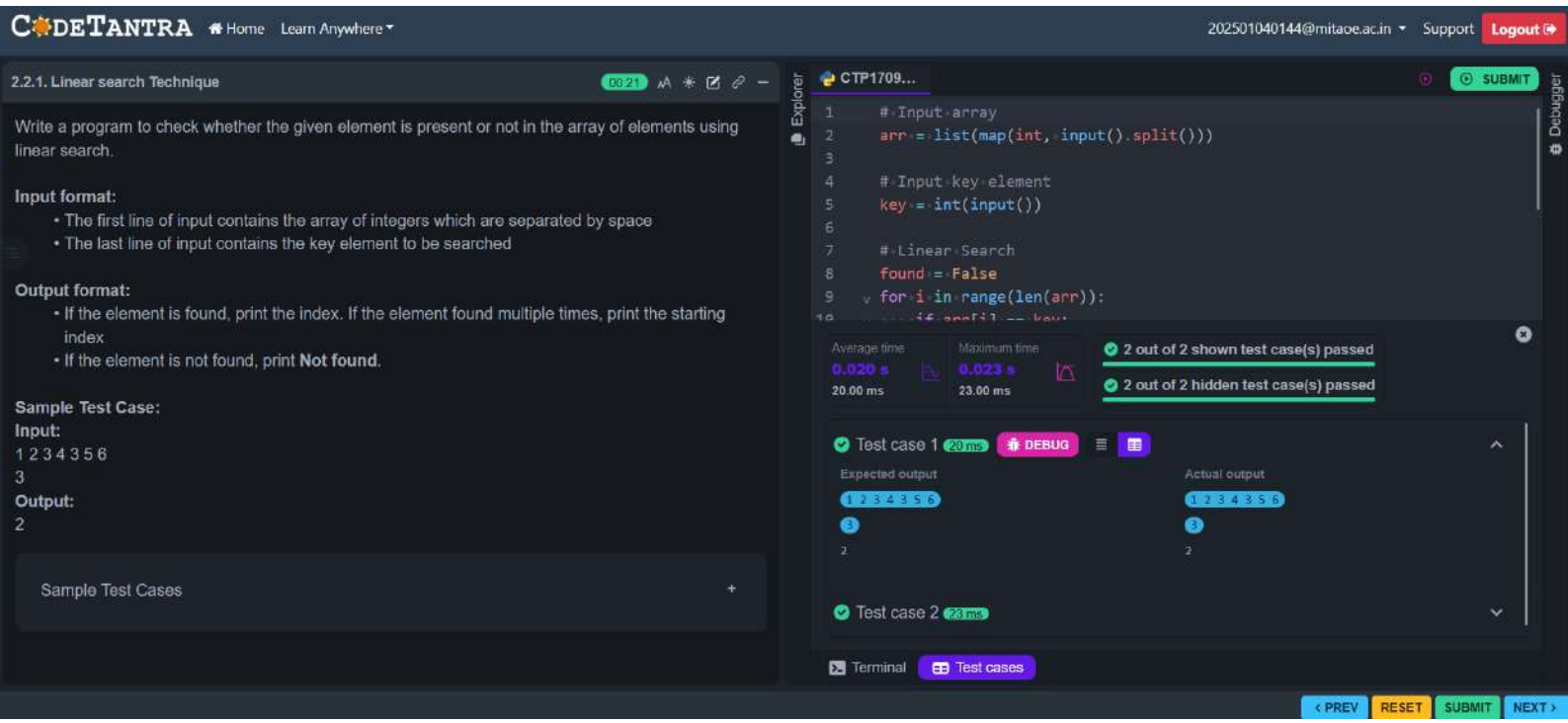

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
1.2.2. Fibonacci Series
07:28
Write a Python program that uses recursion to print the first n terms of the Fibonacci series.
Input Format:
• A single integer n representing the number of terms to generate.
Output Format:
• A single line containing the first n terms of the Fibonacci sequence, separated by spaces.
Sample Test Cases
Explorer
fibonacci....
SUBMIT
Debugger
def fibonacci(n):
if n == 1:
return 0
elif n == 2:
return 1
else:
return fibonacci(n-1) + fibonacci(n - 2)
Average time
0.016 s
16.25 ms
Maximum time
0.035 s
35.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
13 ms
DEBUG
Expected output
5
0 1 1 2 3
Actual output
5
0 1 1 2 3
Test case 2
8 ms
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

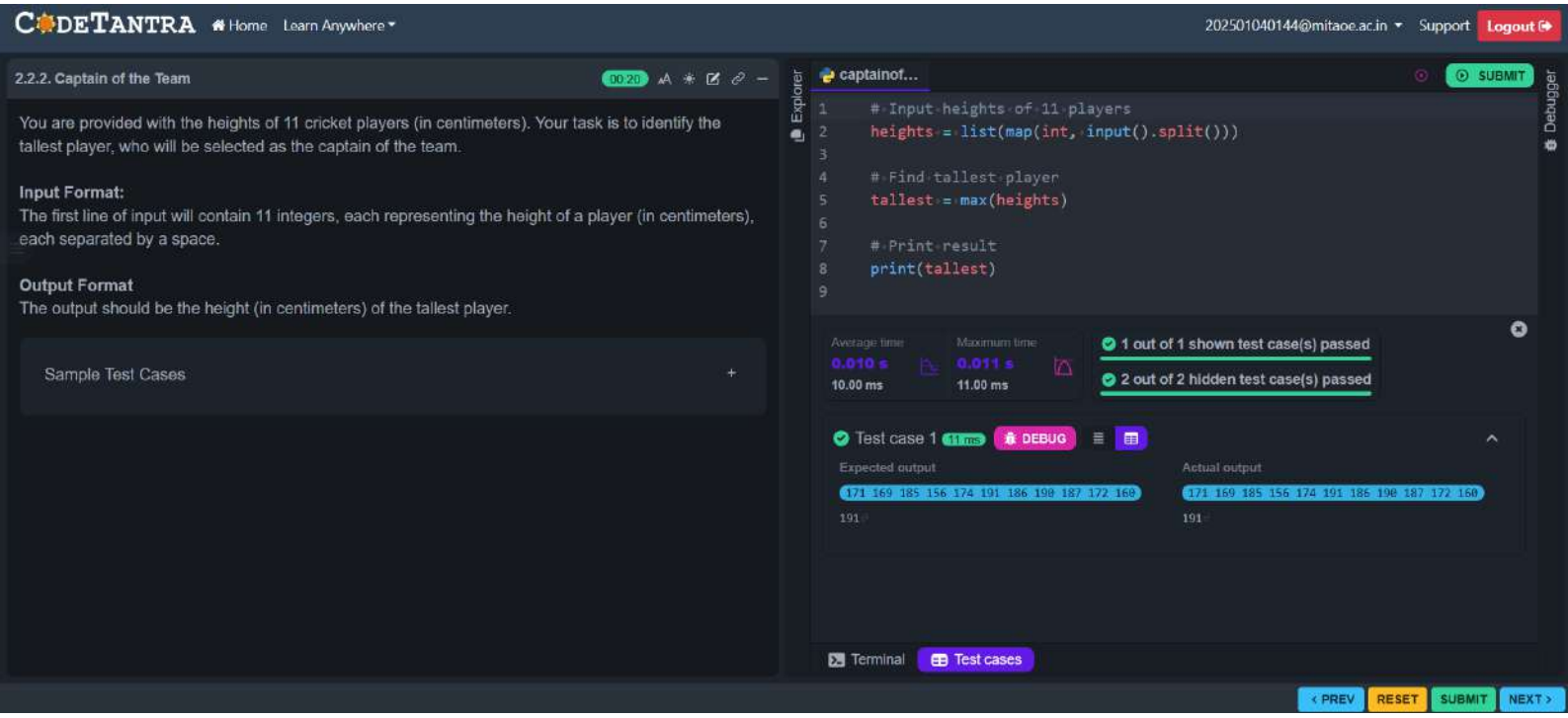

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
1.2.3. Pattern - 1
00:33
Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.
Input Format:
• The input is an integer, representing the number of rows in the pattern.
Output Format
• The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.
Note: Refer to the displayed test cases for the sample pattern.
Sample Test Cases
Explorer
rightangl...
SUBMIT
Debugger
1 # Take input
2 n = int(input())
3
4 # Print right-angled triangle pattern
5 for i in range(1, n + 1):
6 print("* " * i)
7
Average time
0.010 s
9.83 ms
Maximum time
0.018 s
16.00 ms
2 out of 2 shown test case(s) passed
4 out of 4 hidden test case(s) passed
Test case 1
11 ms
DEBUG
Expected output
Actual output
Terminal
Test cases
Show desktop
PREV
RESET

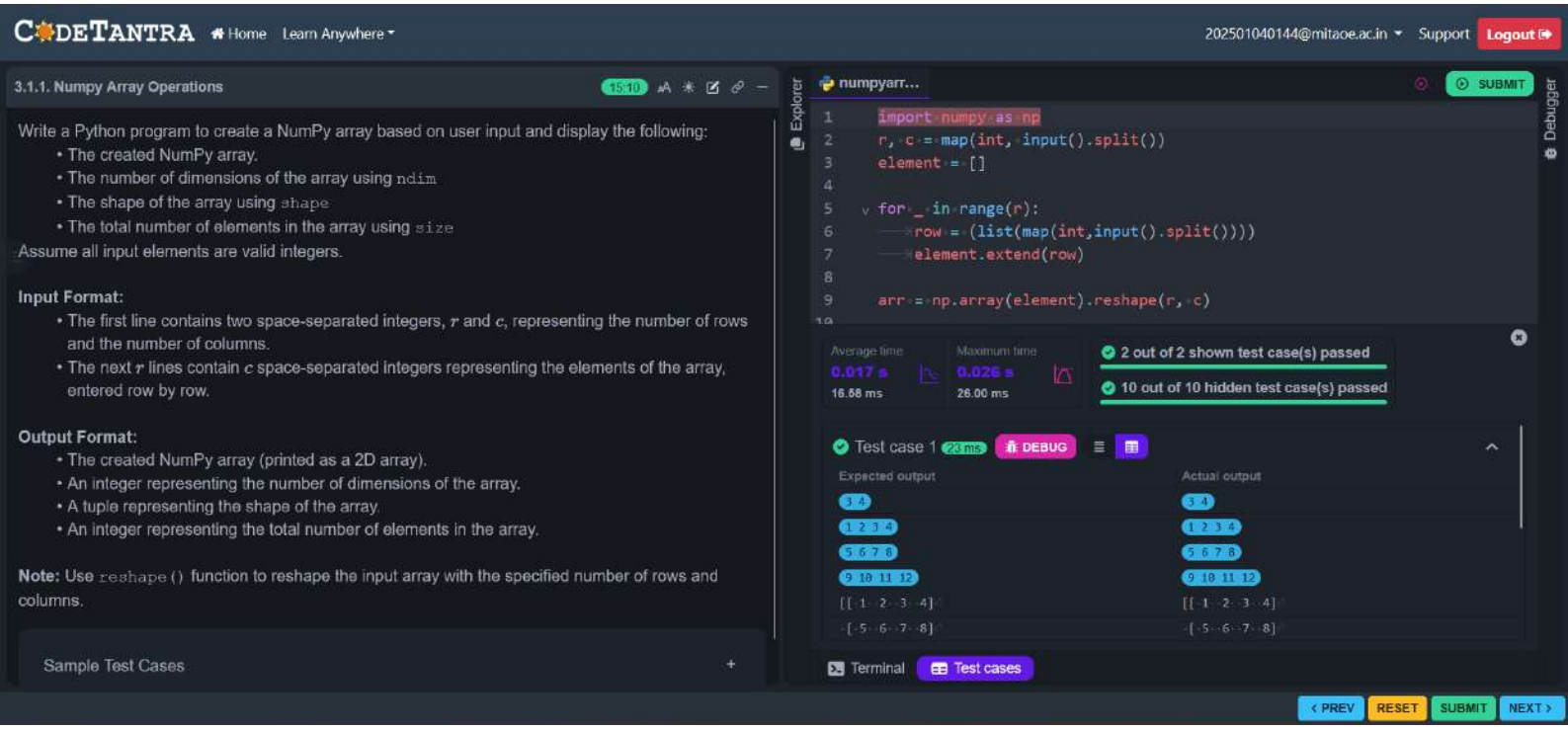

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
1.2.4. Pattern - 2
00:18
Write a Python program to print a right-angled triangle pattern of numbers.
Input Format:
• The input is an integer, representing the number of rows in the pattern.
Output Format:
• The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.
Note:
• Refer to the displayed test cases for the sample pattern.
Sample Test Cases
numberP...
SUBMIT
Explorer
Debugger
# Take input
n = int(input())
# Print number pattern
for i in range(1, n + 1):
for j in range(1, i + 1):
print(j, end=" ")
print()
Average time
0.015 s
14.75 ms
Maximum time
0.016 s
16.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
16 ms
DEBUG
Expected output
Actual output
5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

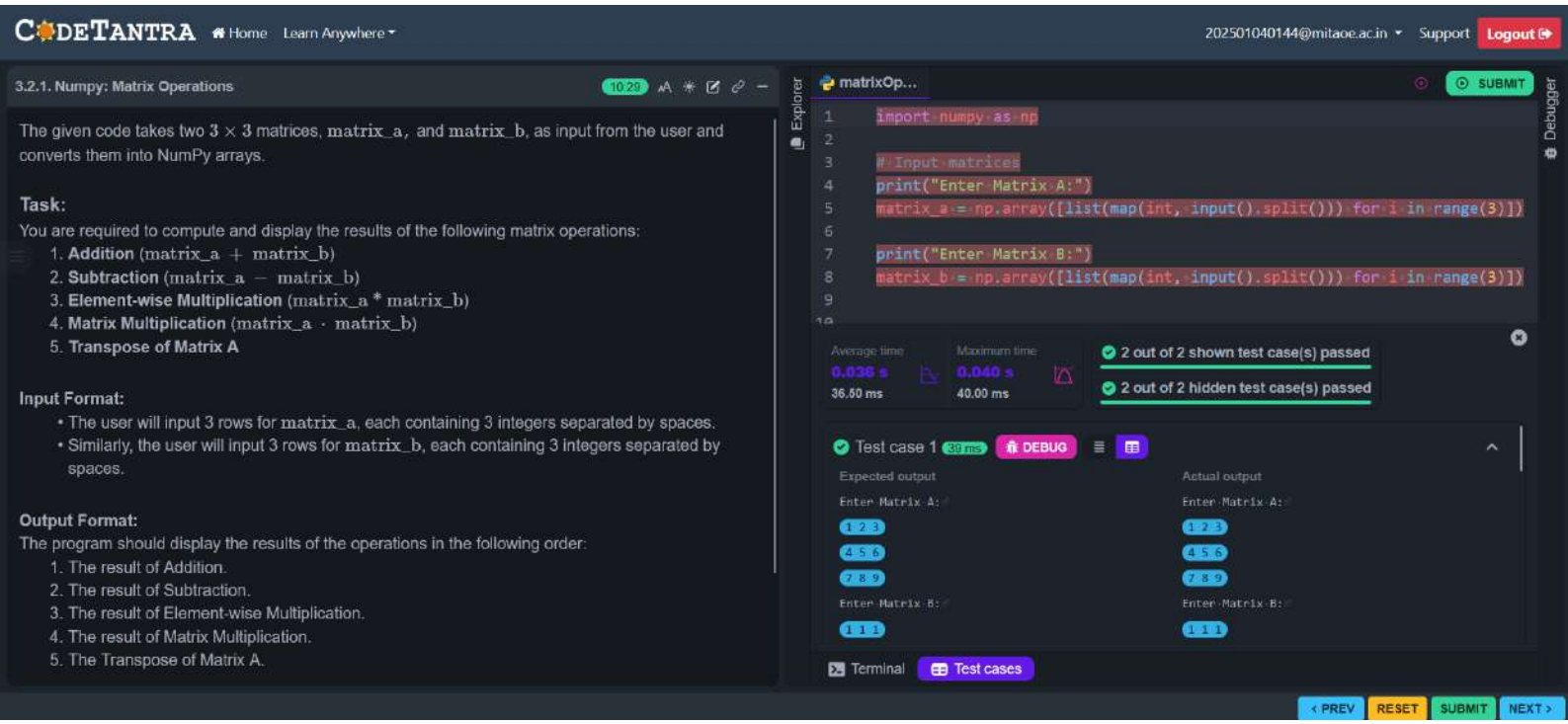

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
2.1.1. List operations
49.03
Write a Python program that implements a menu-driven interface for managing a list of integers. The program should have the following menu options:
1. Add
2. Remove
3. Display
4. Quit
The program should repeatedly prompt the user to enter a choice from the menu. Depending on the choice selected, the program should perform the following actions:
• Add: Prompts the user to enter an integer and add it to the integer list. If the input is not a valid integer, display "Invalid input".
• Remove: Prompts the user to enter an integer to remove from the list. If the integer is found in the list, remove it; otherwise, display "Element not found". If the list is empty, display "List is empty".
• Display: Displays the current list of integers. If the list is empty, display "List is empty".
• Quit: Exits the program.
• The program should handle invalid menu choices by displaying "Invalid choice".Ensure that the program continues to prompt the user until they choose to quit (option 4).
Sample Test Cases
Explorer
listOps.py
SUBMIT
Debugger
numbers = []
while True:
print("1. Add")
print("2. Remove")
print("3. Display")
print("4. Quit")
choice = input("Enter choice: ")
Average time
0.107 s
107.00 ms
Maximum time
0.183 s
183.00 ms
2 out of 2 shown test case(s) passed
1 out of 1 hidden test case(s) passed
Test case 1
69 ms
DEBUG
Expected output
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 11
Actual output
1. Add
2. Remove
3. Display
4. Quit
Enter choice: 1
Integer: 11
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

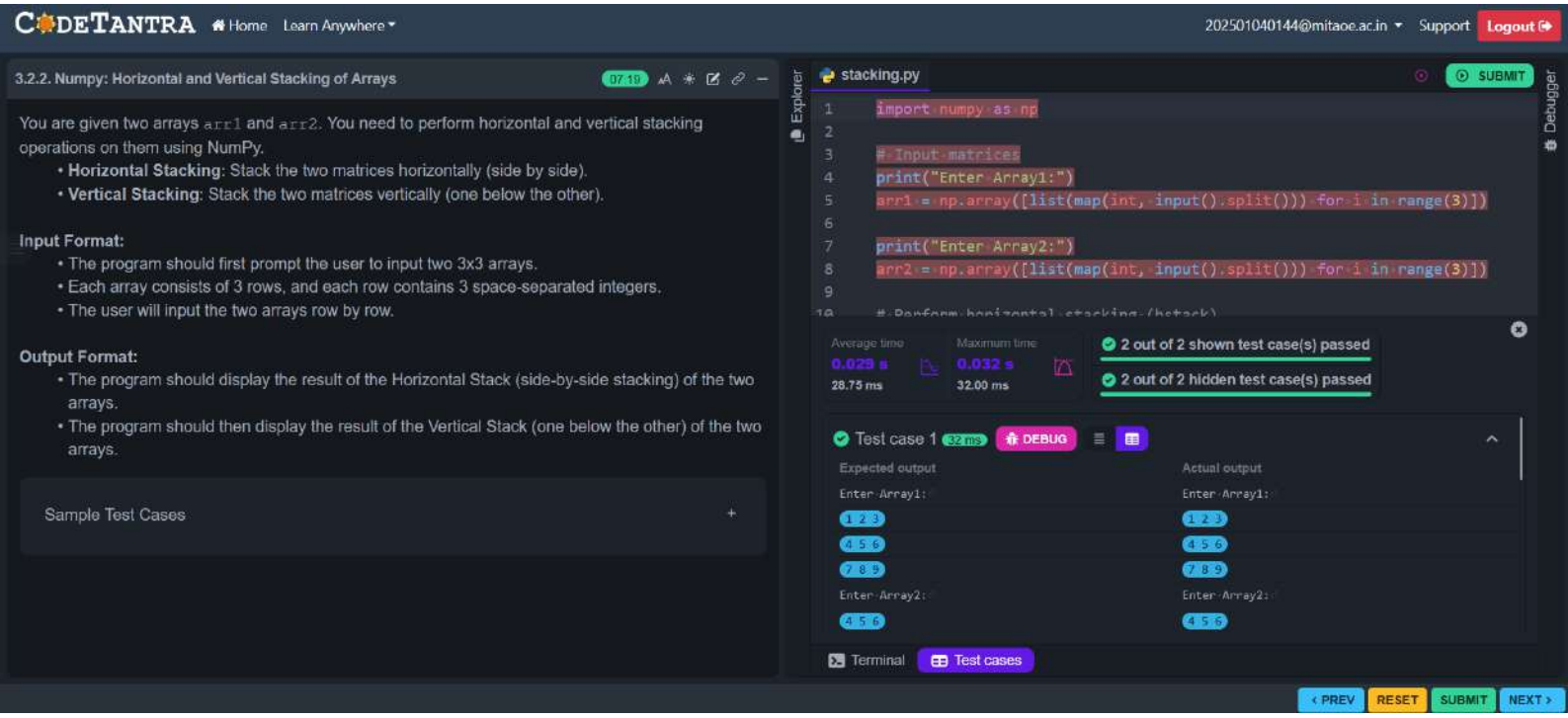

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
2.1.2. Dictionary Operations
05:53
Write a Python program to perform insertion, update, deletion, and traversal operations on a dictionary. An initial dictionary containing 10 predefined records is already given in the program.
Operations to be Performed:
1. Insertion – Insert a new key-value pair into the dictionary using user input.
2. Update – Update the value of an existing key using user input.
3. Deletion – Delete a specified key from the dictionary using user input.
4. Traversal – Traverse the final dictionary and display all key-value pairs.
Input and Output Format:
1. Read an integer representing the key to be inserted and a string representing its value. Insert this new key-value pair into the dictionary and display the dictionary after insertion.
2. Read an integer representing the key to be updated and a string representing the new value. Update the value of the specified key (only if it exists) and display the dictionary after the update.
3. Read an integer representing the key to be deleted. Delete the specified key from the dictionary (only if it exists) and display the dictionary after deletion.
4. Finally, traverse the dictionary and display all key-value pairs.
The program should also display the original dictionary before performing any operations. Each output should be printed with appropriate messages.
Note:
• All operations must be performed using dictionary methods.
• Perform deletion only if possible; leave the dictionary unchanged.
• Refer to the visible test cases for better understanding and strictly match with the input/outputs.
Explorer
DictOper...
SUBMIT
Debugger
# Initial dictionary with 10 predefined records
student = {
1: "Amit",
2: "Riya",
3: "Kiran",
4: "Neha",
5: "Arjun",
6: "Pooja",
7: "Rahul",
Average time
0.053 s
53.25 ms
Maximum time
0.064 s
64.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
49 ms
DEBUG
Expected output
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kira
Actual output
Original Dictionary: {1: 'Amit', 2: 'Riya', 3: 'Kiran', 4: 'Neha', 5: 'Arjun', 6: 'Pooja', 7: 'Rahul', 8: 'Sneha', 9: 'Vikram', 10: 'Anjali'}
11
Suresh
After Insertion: {1: 'Amit', 2: 'Riya', 3: 'Kir
Terminal
Test cases
PREV
RESET
SUBMIT
NEXT

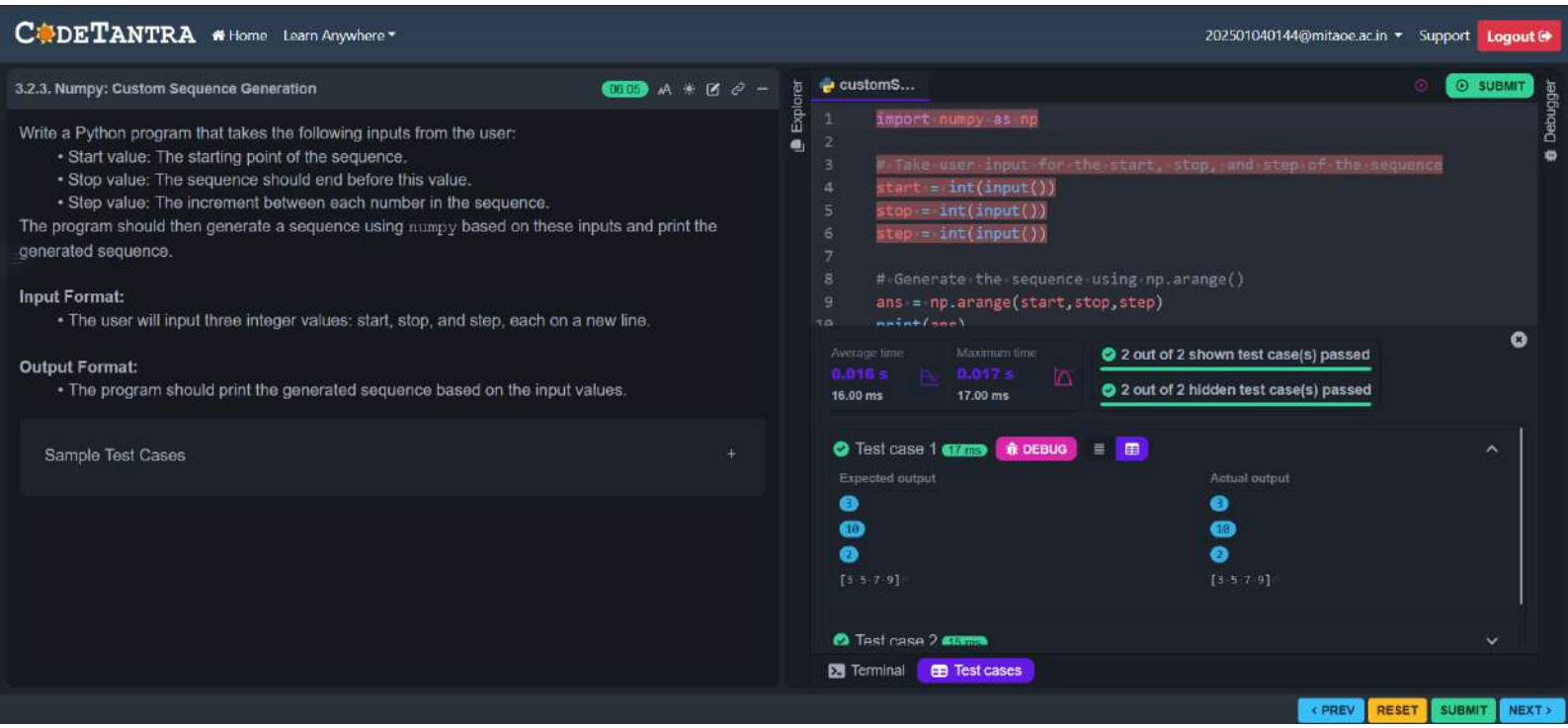

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
2.2.1. Linear search Technique
00:21
Write a program to check whether the given element is present or not in the array of elements using linear search.
Input format:
• The first line of input contains the array of integers which are separated by space
• The last line of input contains the key element to be searched
Output format:
• If the element is found, print the index. If the element found multiple times, print the starting index
• If the element is not found, print Not found.
Sample Test Case:
Input:
1 2 3 4 3 5 6
3
Output:
2
Sample Test Cases
Explorer
CTP1709...
SUBMIT
Debugger
1 # Input array
2 arr = list(map(int, input().split()))
3
4 # Input key element
5 key = int(input())
6
7 # Linear Search
8 found = False
9 for i in range(len(arr)):
Average time
0.020 s
20.00 ms
Maximum time
0.023 s
23.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
20 ms
DEBUG
Expected output
1 2 3 4 3 5 6
3
2
Actual output
1 2 3 4 3 5 6
3
2
Test case 2
23 ms
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

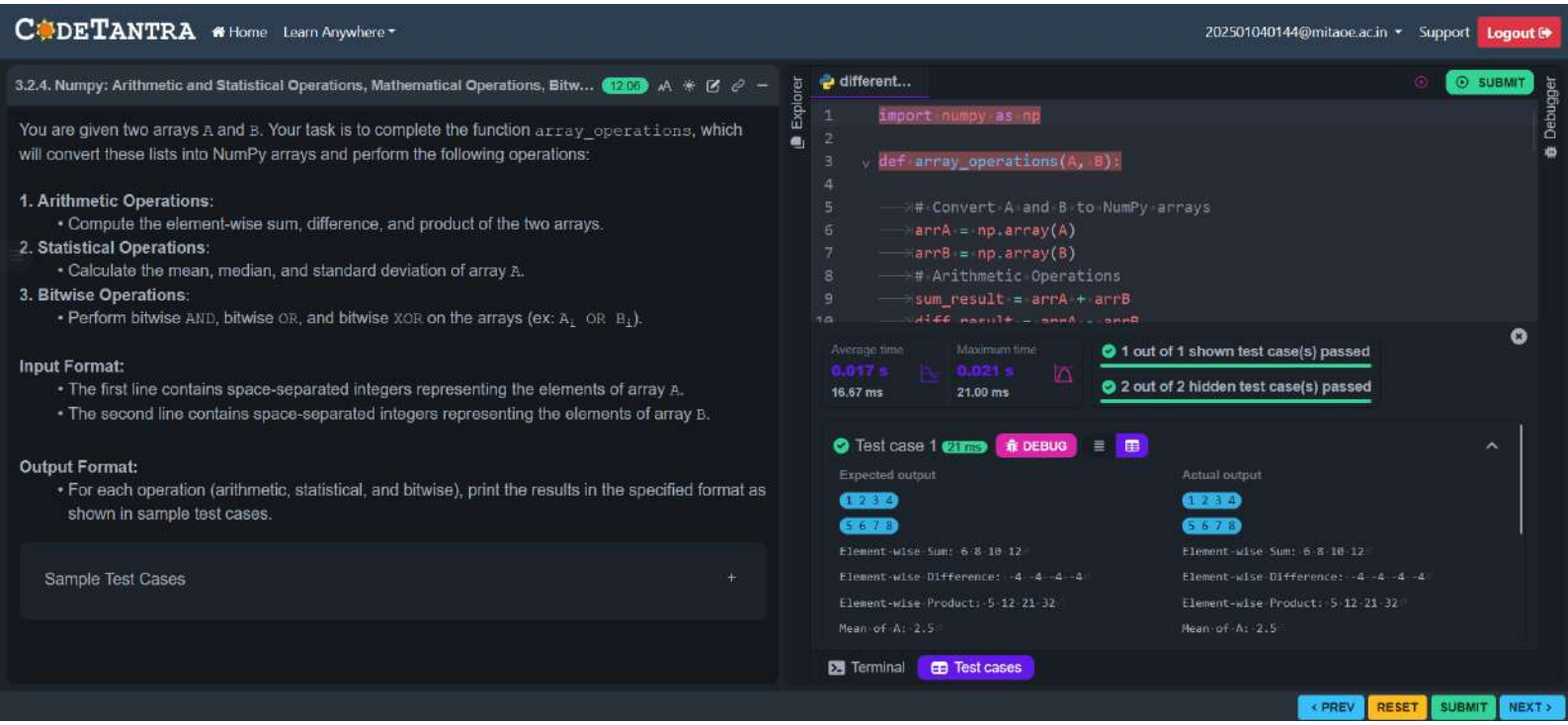

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
2.2.2. Captain of the Team
00:20
You are provided with the heights of 11 cricket players (in centimeters). Your task is to identify the tallest player, who will be selected as the captain of the team.
Input Format:
The first line of input will contain 11 integers, each representing the height of a player (in centimeters), each separated by a space.
Output Format
The output should be the height (in centimeters) of the tallest player.
Sample Test Cases
Explorer
captainof...
SUBMIT
Debugger
# Input heights of 11 players
heights = list(map(int, input().split()))
# Find tallest player
tallest = max(heights)
# Print result
print(tallest)
Average time
0.010 s
10.00 ms
Maximum time
0.011 s
11.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
11 ms
DEBUG
Expected output
171 169 185 156 174 191 186 190 187 172 160
191
Actual output
171 169 185 156 174 191 186 190 187 172 160
191
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

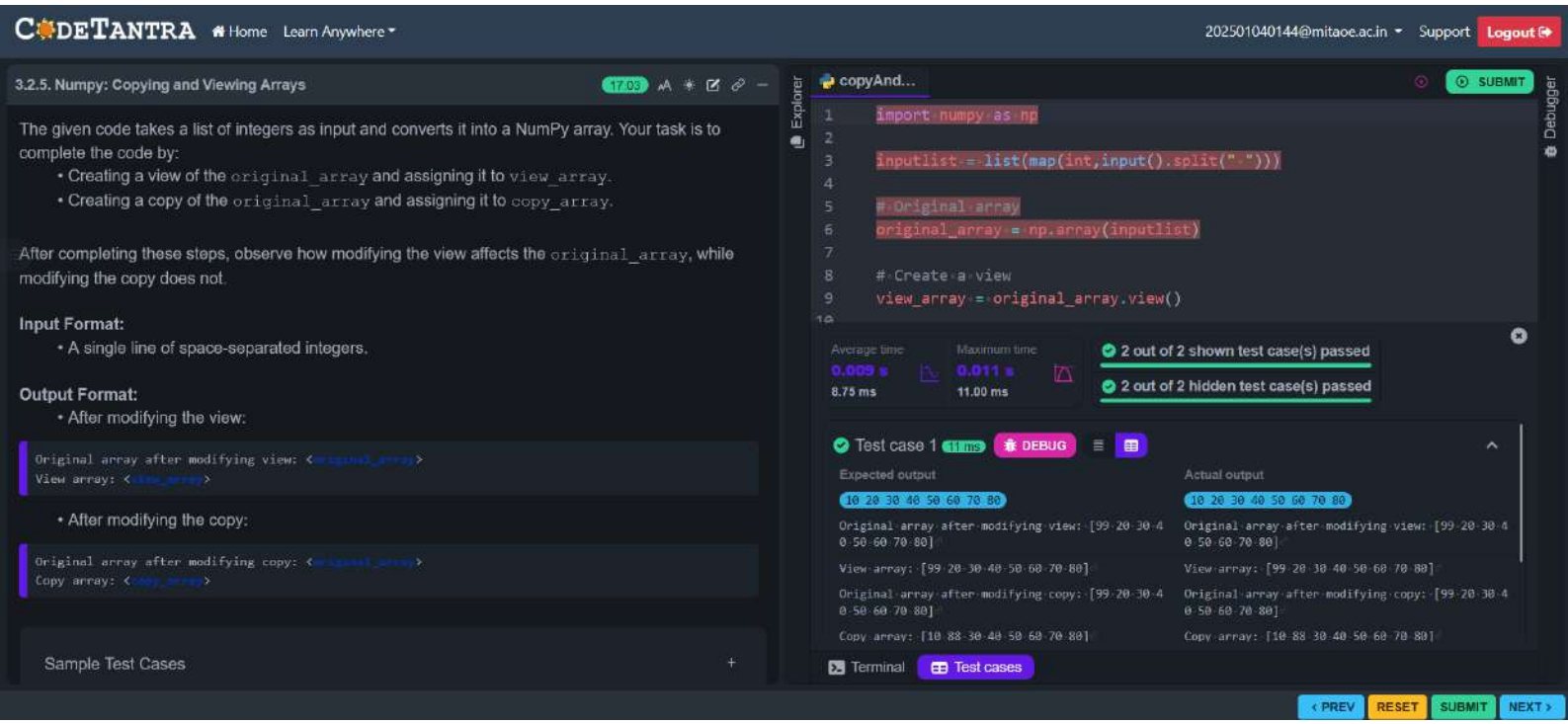

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
3.1.1. Numpy Array Operations
15:10
Write a Python program to create a NumPy array based on user input and display the following:
• The created NumPy array.
• The number of dimensions of the array using ndim
• The shape of the array using shape
• The total number of elements in the array using size
Assume all input elements are valid integers.
Input Format:
• The first line contains two space-separated integers, r and c, representing the number of rows and the number of columns.
• The next r lines contain c space-separated integers representing the elements of the array, entered row by row.
Output Format:
• The created NumPy array (printed as a 2D array).
• An integer representing the number of dimensions of the array.
• A tuple representing the shape of the array.
• An integer representing the total number of elements in the array.
Note: Use reshape() function to reshape the input array with the specified number of rows and columns.
Sample Test Cases
Explorer
numpyarr...
SUBMIT
Debugger
import numpy as np
r, c = map(int, input().split())
element = []
for _ in range(r):
row = (list(map(int,input().split())))
element.extend(row)
arr = np.array(element).reshape(r, c)
Average time
0.017 s
16.58 ms
Maximum time
0.026 s
26.00 ms
2 out of 2 shown test case(s) passed
10 out of 10 hidden test case(s) passed
Test case 1
23 ms
DEBUG
Expected output
3 4
1 2 3 4
5 6 7 8
9 10 11 12
[[1 2 3 4]
[5 6 7 8]]
Actual output
3 4
1 2 3 4
5 6 7 8
9 10 11 12
[[1 2 3 4]
[5 6 7 8]]
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

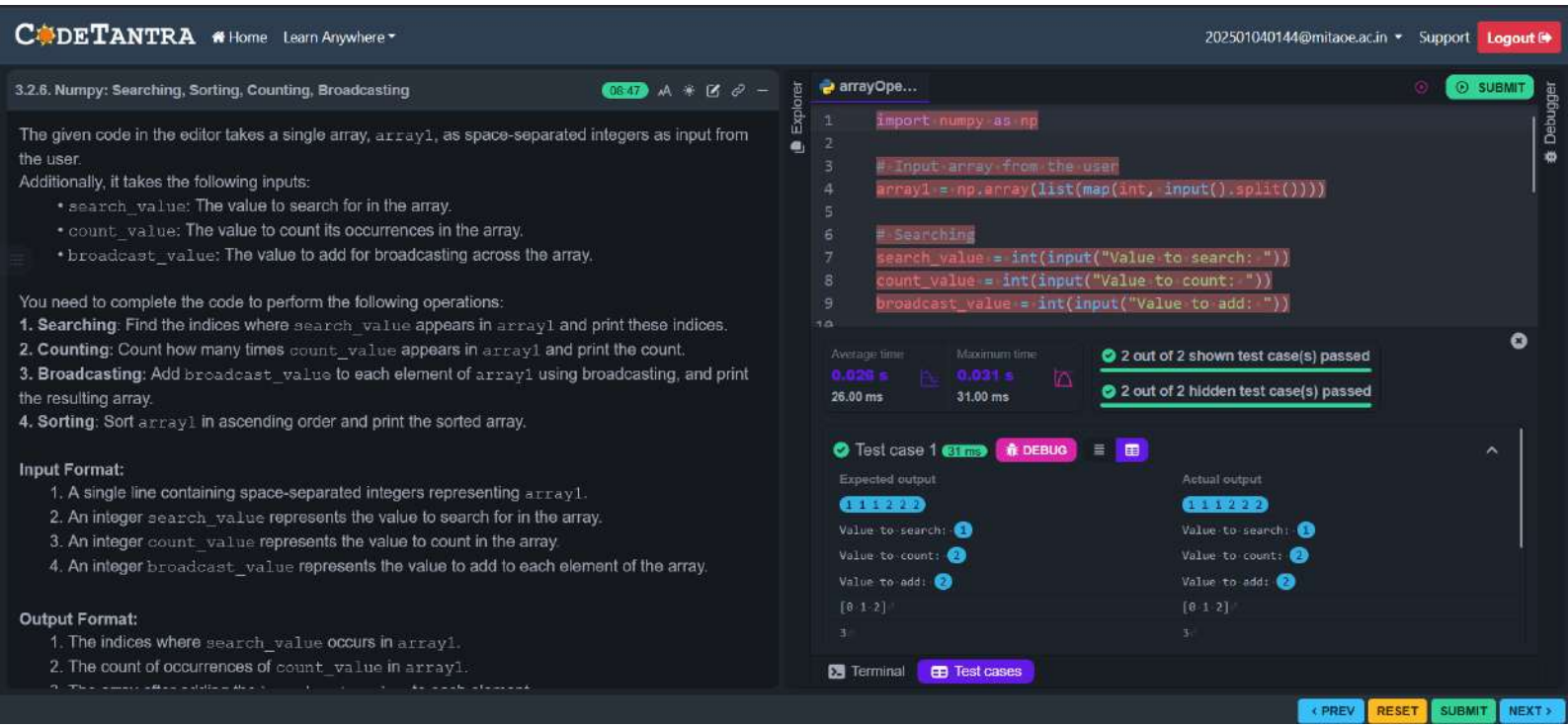

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
3.2.1. Numpy: Matrix Operations
10:20
The given code takes two 3 × 3 matrices, matrix_a, and matrix_b, as input from the user and converts them into NumPy arrays.
Task:
You are required to compute and display the results of the following matrix operations:
1. Addition (matrix_a + matrix_b)
2. Subtraction (matrix_a − matrix_b)
3. Element-wise Multiplication (matrix_a * matrix_b)
4. Matrix Multiplication (matrix_a · matrix_b)
5. Transpose of Matrix A
Input Format:
• The user will input 3 rows for matrix_a, each containing 3 integers separated by spaces.
• Similarly, the user will input 3 rows for matrix_b, each containing 3 integers separated by spaces.
Output Format:
The program should display the results of the operations in the following order:
1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.
Explorer
matrixOp...
SUBMIT
Debugger
import numpy as np
# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
Average time
0.036 s
36.50 ms
Maximum time
0.040 s
40.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
39 ms
DEBUG
Expected output
Enter Matrix A:
1 2 3
4 5 6
7 8 9
Enter Matrix B:
1 1 1
Actual output
Enter Matrix A:
1 2 3
4 5 6
7 8 9
Enter Matrix B:
1 1 1
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

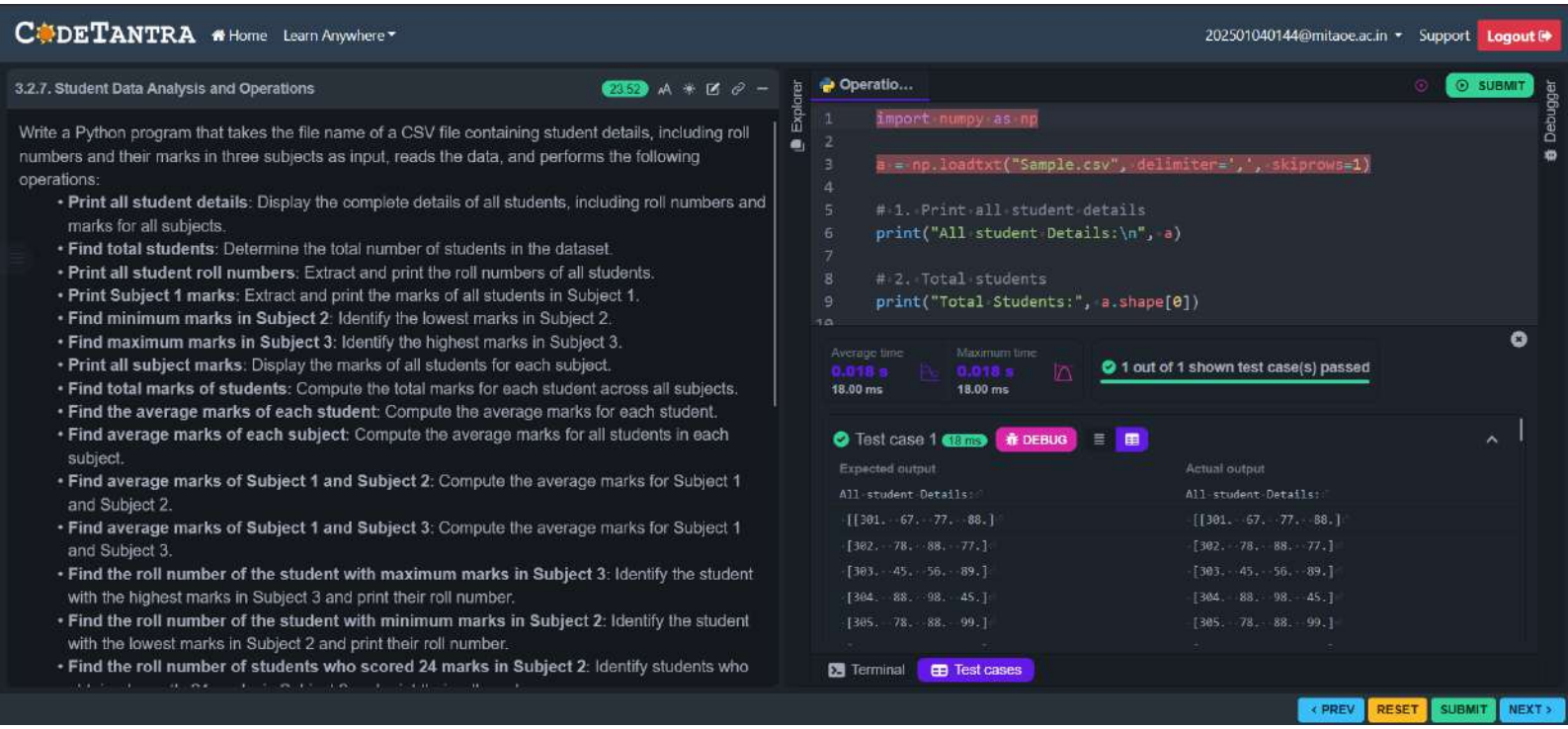

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays
07:19
You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.
• Horizontal Stacking: Stack the two matrices horizontally (side by side).
• Vertical Stacking: Stack the two matrices vertically (one below the other).
Input Format:
• The program should first prompt the user to input two 3x3 arrays.
• Each array consists of 3 rows, and each row contains 3 space-separated integers.
• The user will input the two arrays row by row.
Output Format:
• The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
• The program should then display the result of the Vertical Stack (one below the other) of the two arrays.
Sample Test Cases
Explorer
stacking.py
SUBMIT
Debugger
import numpy as np
# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])
print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])
Average time
0.029 s
28.75 ms
Maximum time
0.032 s
32.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
32 ms
DEBUG
Expected output
Enter Array1:
1 2 3
4 5 6
7 8 9
Enter Array2:
4 5 6
Actual output
Enter Array1:
1 2 3
4 5 6
7 8 9
Enter Array2:
4 5 6
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

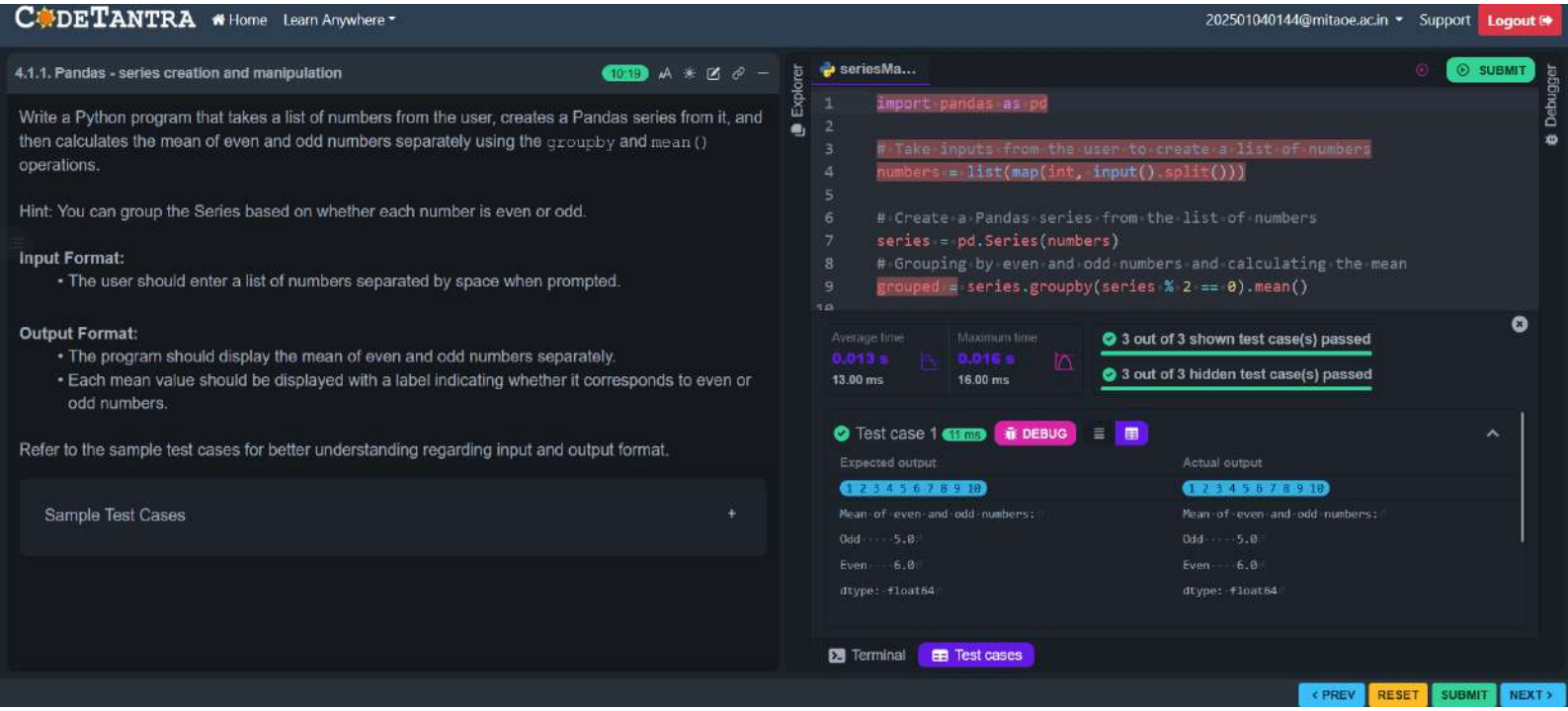

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
3.2.3. Numpy: Custom Sequence Generation
06:05
Write a Python program that takes the following inputs from the user:
• Start value: The starting point of the sequence.
• Stop value: The sequence should end before this value.
• Step value: The increment between each number in the sequence.
The program should then generate a sequence using numpy based on these inputs and print the generated sequence.
Input Format:
• The user will input three integer values: start, stop, and step, each on a new line.
Output Format:
• The program should print the generated sequence based on the input values.
Sample Test Cases
Explorer
customS...
SUBMIT
Debugger
import numpy as np
# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())
# Generate the sequence using np.arange()
ans = np.arange(start,stop,step)
print(ans)
Average time
0.016 s
16.00 ms
Maximum time
0.017 s
17.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
17 ms
DEBUG
Expected output
Actual output
3
10
2
[3 5 7 9]
Test case 2
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

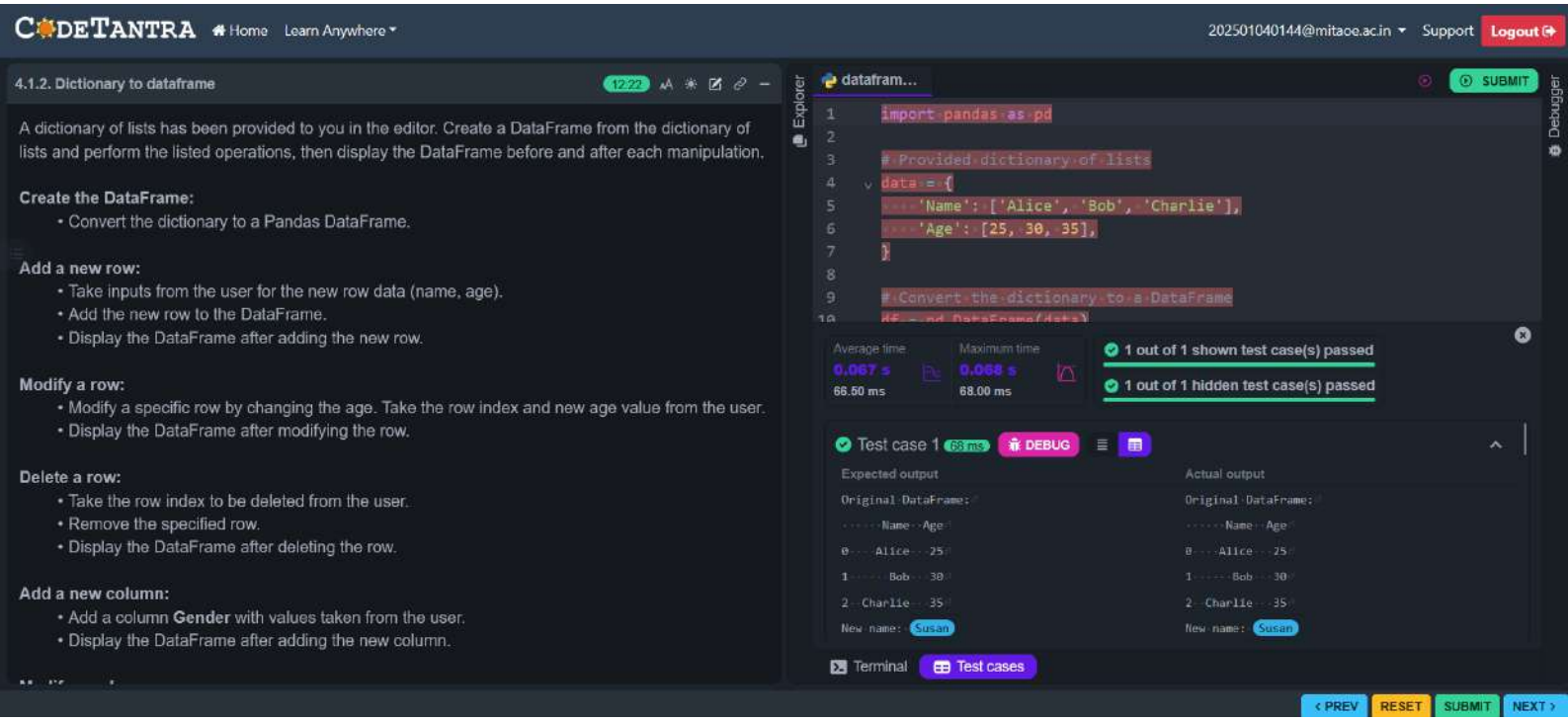

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitw...
12:06
You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:
1. Arithmetic Operations:
• Compute the element-wise sum, difference, and product of the two arrays.
2. Statistical Operations:
• Calculate the mean, median, and standard deviation of array A.
3. Bitwise Operations:
• Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A1 OR B1).
Input Format:
• The first line contains space-separated integers representing the elements of array A.
• The second line contains space-separated integers representing the elements of array B.
Output Format:
• For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.
Sample Test Cases
Explorer
different...
SUBMIT
Debugger
import numpy as np
def array_operations(A, B):
# Convert A and B to NumPy arrays
arrA = np.array(A)
arrB = np.array(B)
# Arithmetic Operations
sum_result = arrA + arrB
Average time
0.017 s
16.67 ms
Maximum time
0.021 s
21.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
21 ms
DEBUG
Expected output
1 2 3 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32
Mean of A: 2.5
Actual output
1 2 3 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32
Mean of A: 2.5
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

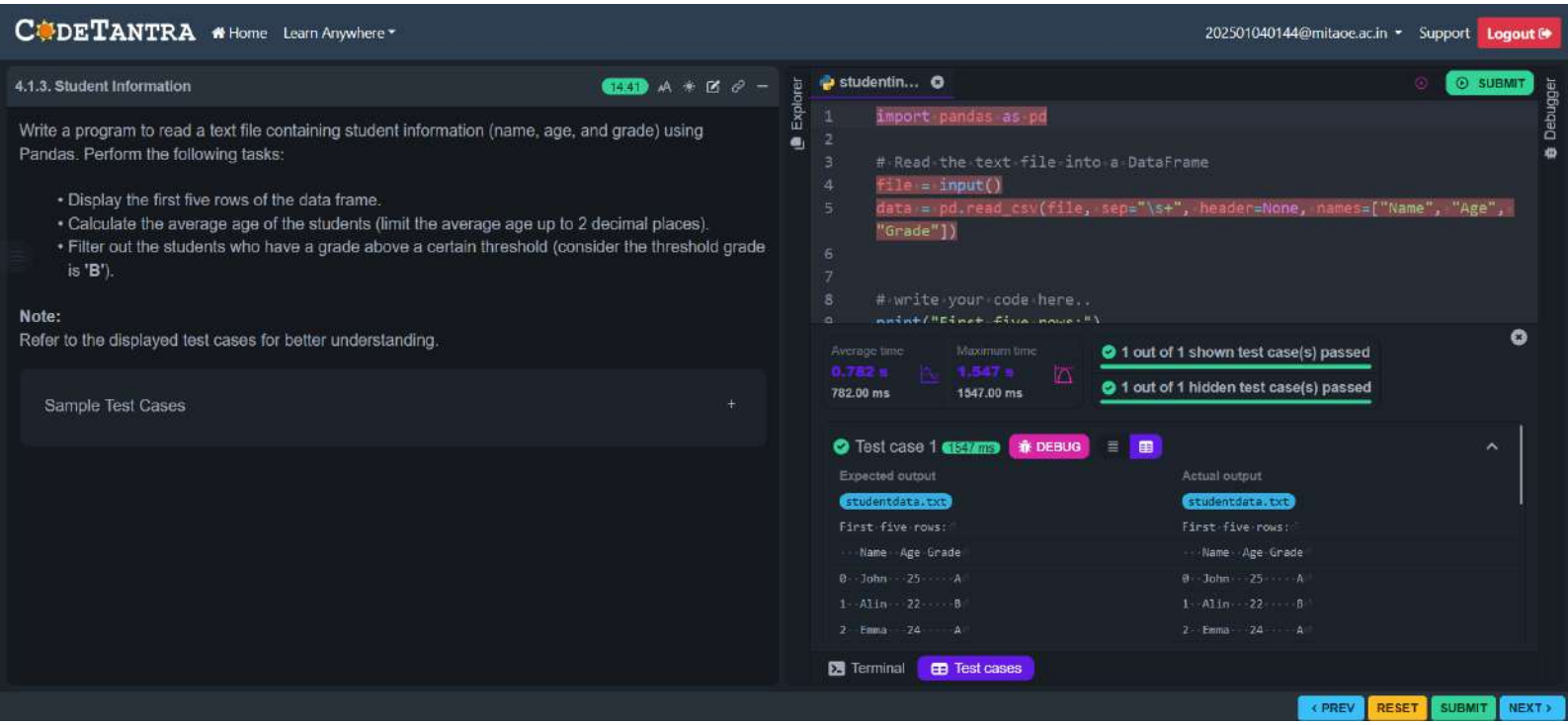

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
3.2.5. Numpy: Copying and Viewing Arrays
17:03
The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:
• Creating a view of the original_array and assigning it to view_array.
• Creating a copy of the original_array and assigning it to copy_array.
After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.
Input Format:
• A single line of space-separated integers.
Output Format:
• After modifying the view:
Original array after modifying view: <original_array>
View array: <view_array>
• After modifying the copy:
Original array after modifying copy: <original_array>
Copy array: <copy_array>
Sample Test Cases
Explorer
copyAnd...
SUBMIT
Debugger
1 import numpy as np
2
3 inputlist = list(map(int,input().split(" ")))
4
5 # Original array
6 original_array = np.array(inputlist)
7
8 # Create a view
9 view_array = original_array.view()
Average time
0.009 s
8.75 ms
Maximum time
0.011 s
11.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
11 ms
DEBUG
Expected output
10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]
View array: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]
Actual output
10 20 30 40 50 60 70 80
Original array after modifying view: [99 20 30 40 50 60 70 80]
View array: [99 20 30 40 50 60 70 80]
Original array after modifying copy: [99 20 30 40 50 60 70 80]
Copy array: [10 88 30 40 50 60 70 80]
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

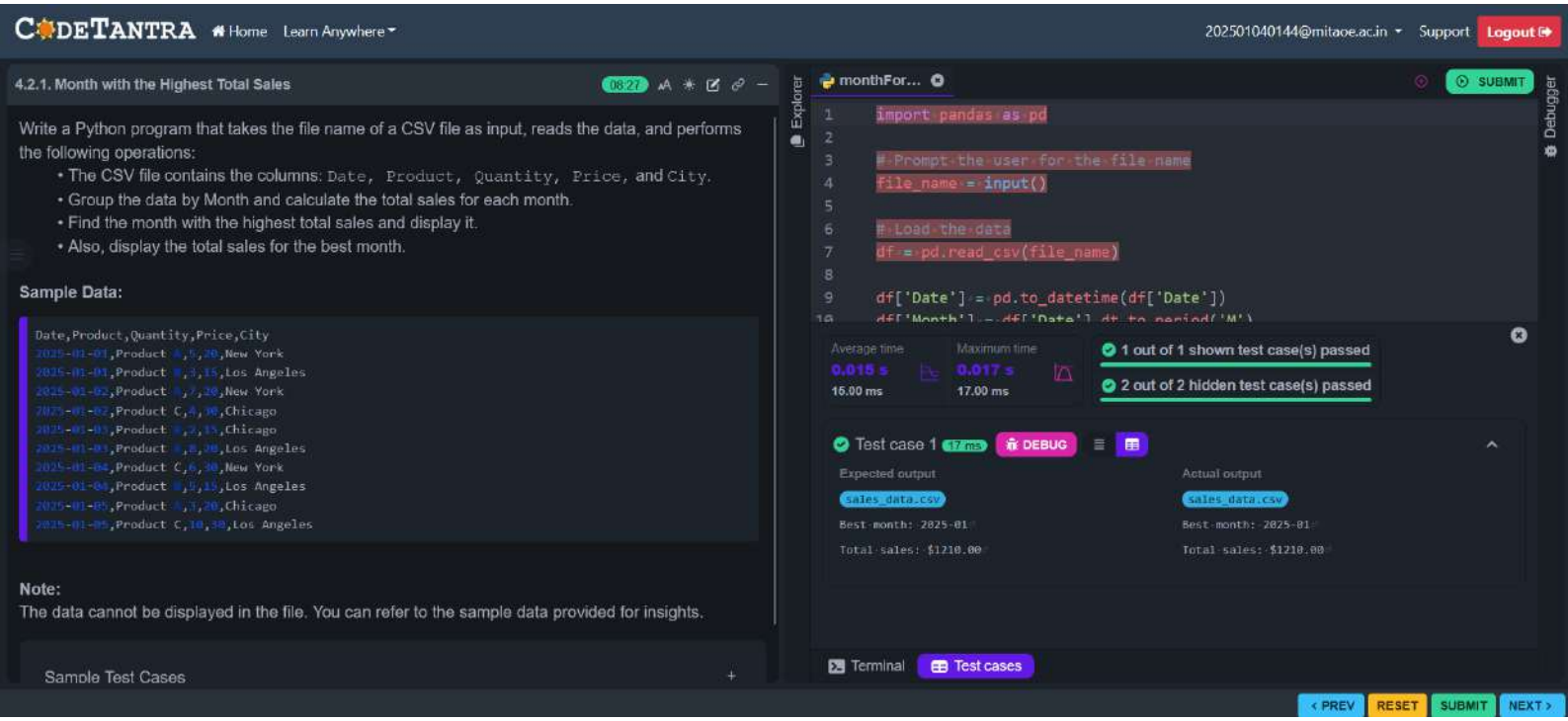

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting
06:47
The given code in the editor takes a single array, array1, as space-separated integers as input from the user.
Additionally, it takes the following inputs:
• search_value: The value to search for in the array.
• count_value: The value to count its occurrences in the array.
• broadcast_value: The value to add for broadcasting across the array.
You need to complete the code to perform the following operations:
1. Searching: Find the indices where search_value appears in array1 and print these indices.
2. Counting: Count how many times count_value appears in array1 and print the count.
3. Broadcasting: Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
4. Sorting: Sort array1 in ascending order and print the sorted array.
Input Format:
1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.
Output Format:
1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
Explorer
arrayOpe...
SUBMIT
Debugger
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
Average time
0.020 s
26.00 ms
Maximum time
0.031 s
31.00 ms
2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
31 ms
DEBUG
Expected output
1 1 1 2 2 2
Value to search: 1
Value to count: 2
Value to add: 2
[0 1 2]
3
Actual output
1 1 1 2 2 2
Value to search: 1
Value to count: 2
Value to add: 2
[0 1 2]
3
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

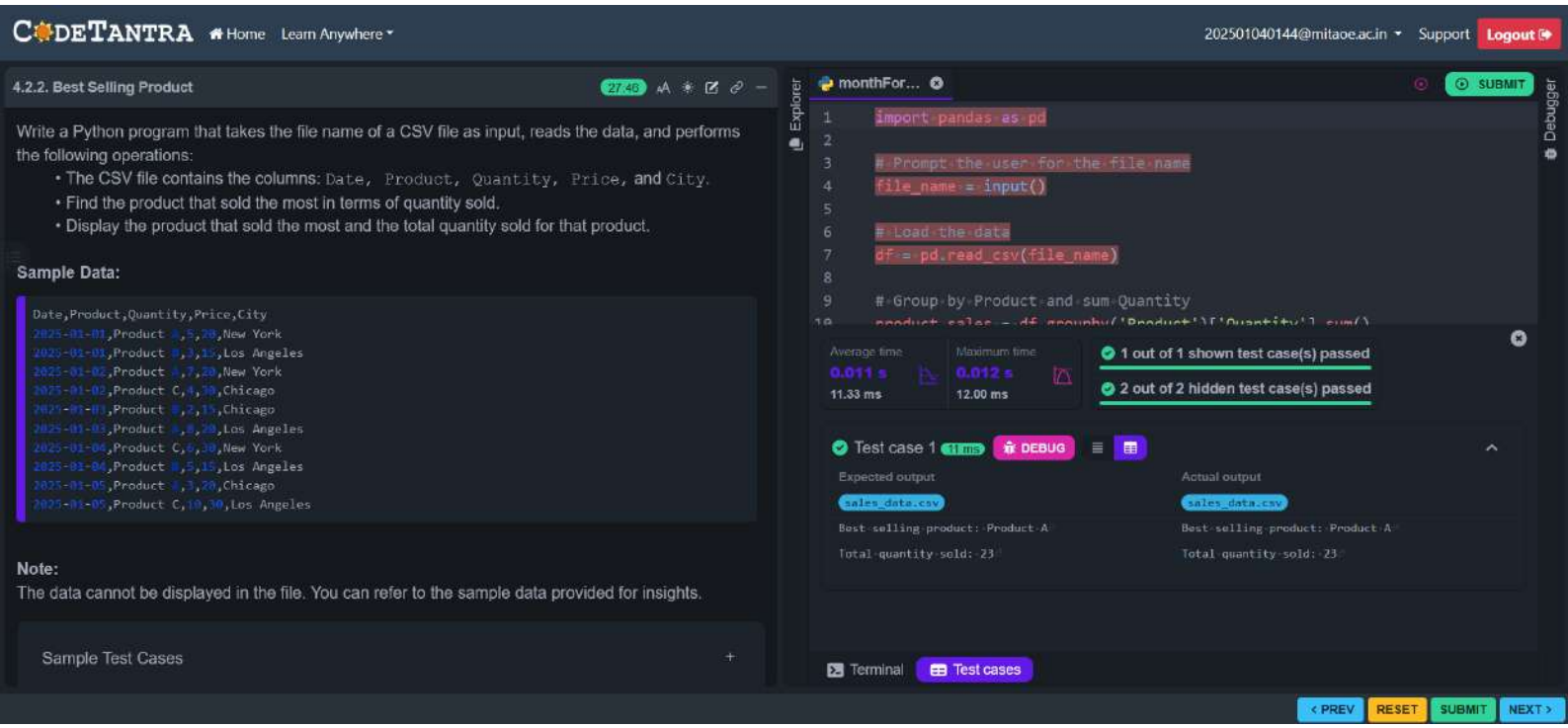

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
3.2.7. Student Data Analysis and Operations
23.52
Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:
• Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
• Find total students: Determine the total number of students in the dataset.
• Print all student roll numbers: Extract and print the roll numbers of all students.
• Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
• Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
• Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
• Print all subject marks: Display the marks of all students for each subject.
• Find total marks of students: Compute the total marks for each student across all subjects.
• Find the average marks of each student: Compute the average marks for each student.
• Find average marks of each subject: Compute the average marks for all students in each subject.
• Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
• Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
• Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
• Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
• Find the roll number of students who scored 24 marks in Subject 2: Identify students who
Explorer
Operatio...
SUBMIT
Debugger
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5 # 1. Print all student details
6 print("All student Details:\n", a)
7
8 # 2. Total students
9 print("Total Students:", a.shape[0])
Average time
0.018 s
18.00 ms
Maximum time
0.018 s
18.00 ms
1 out of 1 shown test case(s) passed
Test case 1
18 ms
DEBUG
Expected output
All student Details:
[[301. 67. 77. 88.]
[302. 78. 88. 77.]
[303. 45. 56. 89.]
[304. 88. 98. 45.]
[305. 78. 88. 99.]
Actual output
All student Details:
[[301. 67. 77. 88.]
[302. 78. 88. 77.]
[303. 45. 56. 89.]
[304. 88. 98. 45.]
[305. 78. 88. 99.]
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

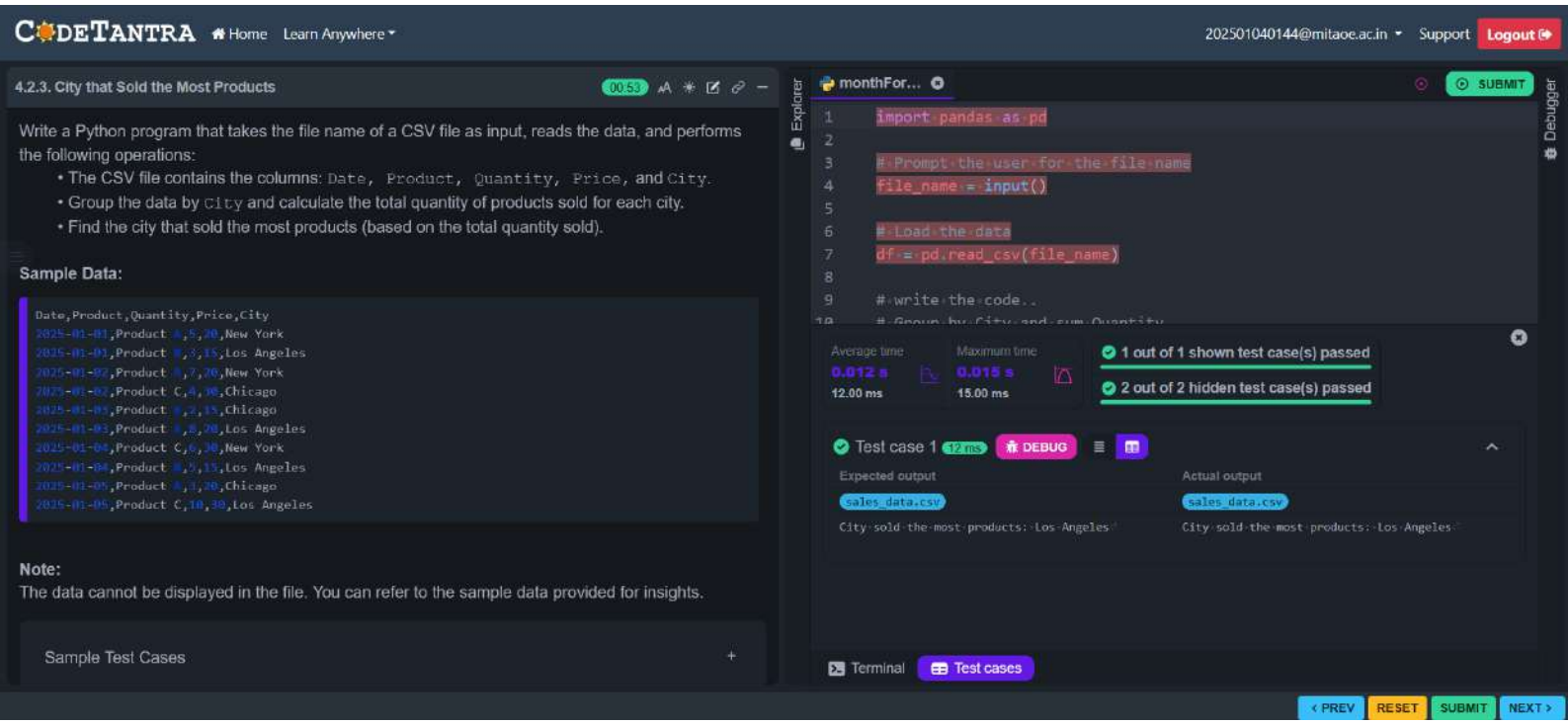

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.1.1. Pandas - series creation and manipulation
10:19
Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.
Hint: You can group the Series based on whether each number is even or odd.
Input Format:
• The user should enter a list of numbers separated by space when prompted.
Output Format:
• The program should display the mean of even and odd numbers separately.
• Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.
Refer to the sample test cases for better understanding regarding input and output format.
Sample Test Cases
Explorer
seriesMa...
SUBMIT
Debugger
import pandas as pd
# Take inputs from the user to create a list of numbers
numbers = list(map(int, input().split()))
# Create a Pandas series from the list of numbers
series = pd.Series(numbers)
# Grouping by even and odd numbers and calculating the mean
grouped = series.groupby(series % 2 == 0).mean()
Average time
0.013 s
13.00 ms
Maximum time
0.016 s
16.00 ms
3 out of 3 shown test case(s) passed
3 out of 3 hidden test case(s) passed
Test case 1
11 ms
DEBUG
Expected output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd 5.0
Even 6.0
dtype: float64
Actual output
1 2 3 4 5 6 7 8 9 10
Mean of even and odd numbers:
Odd 5.0
Even 6.0
dtype: float64
Terminal
Test cases
PREV
RESET
SUBMIT
NEXT

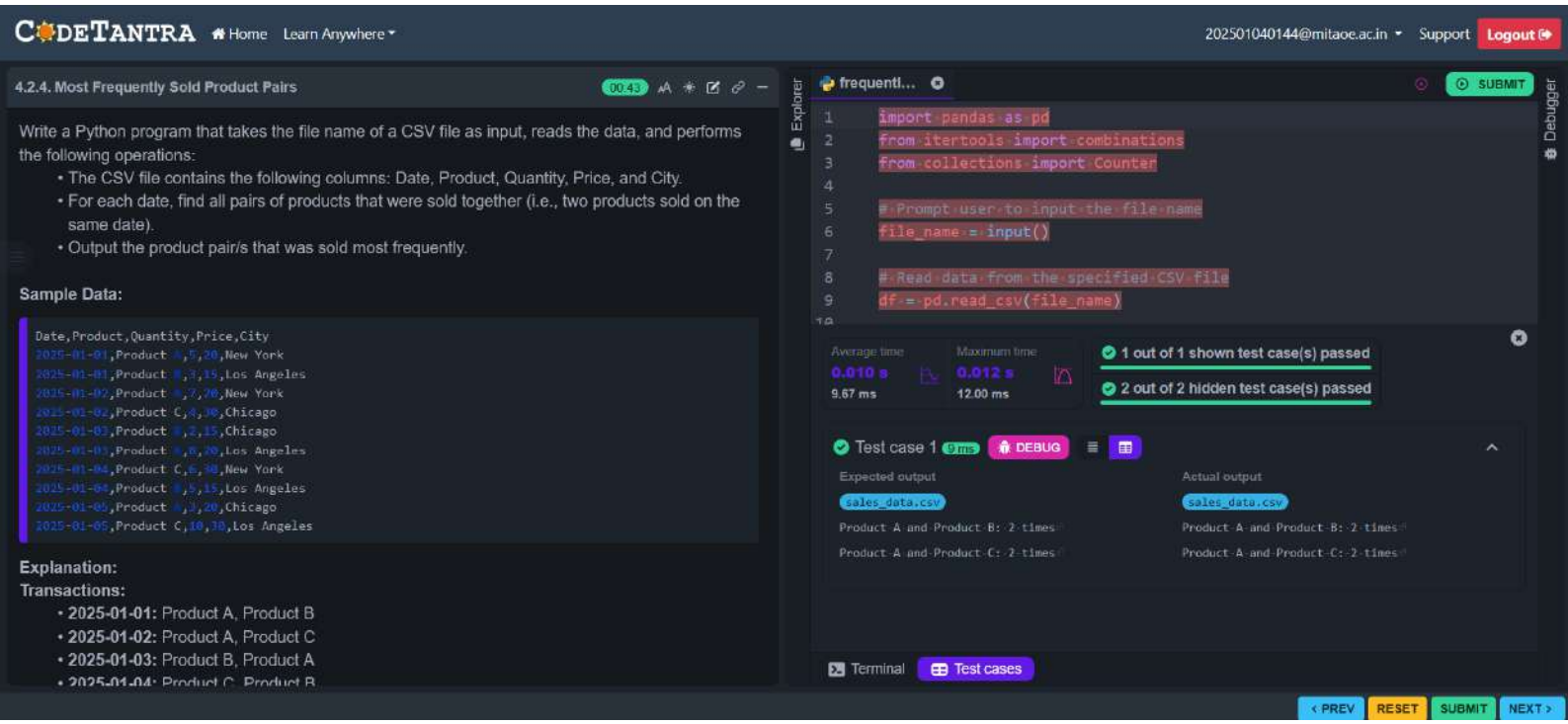

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.1.2. Dictionary to dataframe
12:22
A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.
Create the DataFrame:
• Convert the dictionary to a Pandas DataFrame.
Add a new row:
• Take inputs from the user for the new row data (name, age).
• Add the new row to the DataFrame.
• Display the DataFrame after adding the new row.
Modify a row:
• Modify a specific row by changing the age. Take the row index and new age value from the user.
• Display the DataFrame after modifying the row.
Delete a row:
• Take the row index to be deleted from the user.
• Remove the specified row.
• Display the DataFrame after deleting the row.
Add a new column:
• Add a column Gender with values taken from the user.
• Display the DataFrame after adding the new column.
Explorer
dataframe...
SUBMIT
Debugger
1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5 'Name': ['Alice', 'Bob', 'Charlie'],
6 'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
Average time
0.067 s
66.50 ms
Maximum time
0.068 s
68.00 ms
1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed
Test case 1
68 ms
DEBUG
Expected output
Original DataFrame:
Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
New name: Susan
Actual output
Original DataFrame:
Name Age
0 Alice 25
1 Bob 30
2 Charlie 35
New name: Susan
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

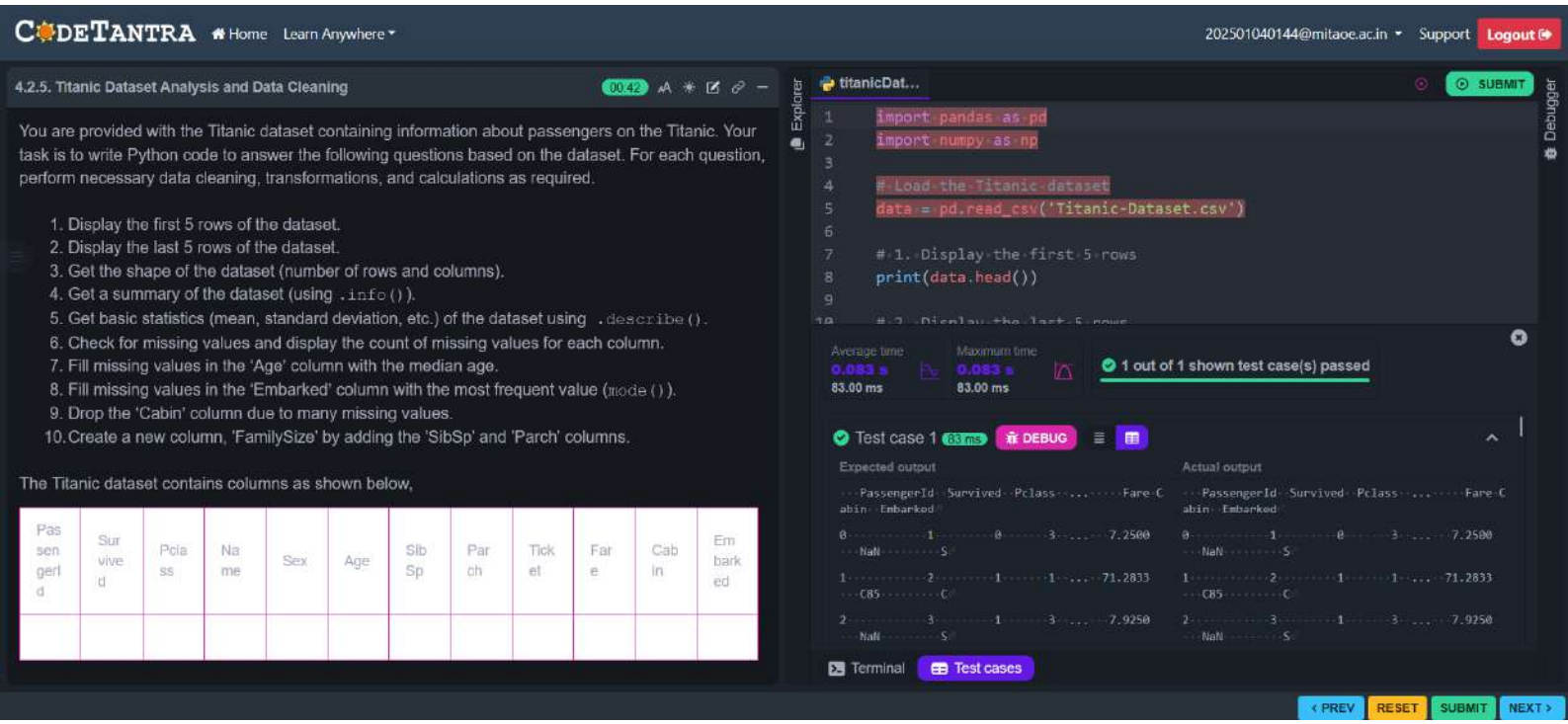

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.1.3. Student Information
14:41
Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:
• Display the first five rows of the data frame.
• Calculate the average age of the students (limit the average age up to 2 decimal places).
• Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').
Note:
Refer to the displayed test cases for better understanding.
Sample Test Cases
Explorer
studentin...
SUBMIT
Debugger
import pandas as pd
# Read the text file into a DataFrame
file = input()
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
# write your code here..
Average time
0.782 s
782.00 ms
Maximum time
1.547 s
1547.00 ms
1 out of 1 shown test case(s) passed
1 out of 1 hidden test case(s) passed
Test case 1
1547 ms
DEBUG
Expected output
studentdata.txt
First five rows:
Name Age Grade
0 John 25 A
1 Alin 22 B
2 Emma 24 A
Actual output
studentdata.txt
First five rows:
Name Age Grade
0 John 25 A
1 Alin 22 B
2 Emma 24 A
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›

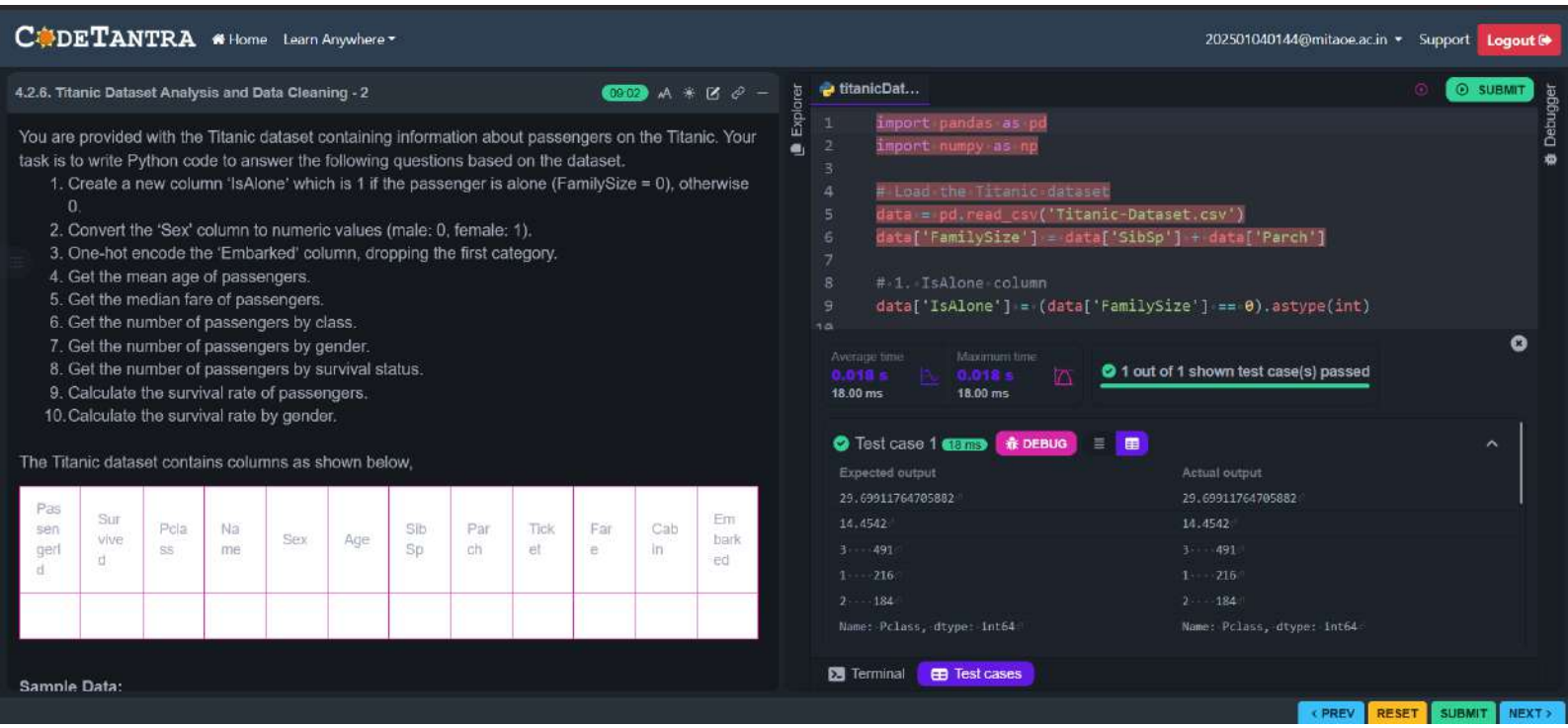

CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.2.1. Month with the Highest Total Sales
08:27
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the columns: Date, Product, Quantity, Price, and City.
• Group the data by Month and calculate the total sales for each month.
• Find the month with the highest total sales and display it.
• Also, display the total sales for the best month.
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product B,8,30,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.
Sample Test Cases
Explorer
monthFor...
SUBMIT
Debugger
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
df['Date'] = pd.to_datetime(df['Date'])
Average time
0.015 s
16.00 ms
Maximum time
0.017 s
17.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
17 ms
DEBUG
Expected output
sales_data.csv
Best month: 2025-01
Total sales: $1210.00
Actual output
sales_data.csv
Best month: 2025-01
Total sales: $1210.00
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >


CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.2.2. Best Selling Product
27:46
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the columns: Date, Product, Quantity, Price, and City.
• Find the product that sold the most in terms of quantity sold.
• Display the product that sold the most and the total quantity sold for that product.
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,50,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.
Sample Test Cases
Explorer
monthFor...
SUBMIT
Debugger
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
# Group by Product and sum Quantity
Average time
0.011 s
11.33 ms
Maximum time
0.012 s
12.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
11 ms
DEBUG
Expected output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23
Actual output
sales_data.csv
Best selling product: Product A
Total quantity sold: 23
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >


CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.2.3. City that Sold the Most Products
00:53
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the columns: Date, Product, Quantity, Price, and City.
• Group the data by City and calculate the total quantity of products sold for each city.
• Find the city that sold the most products (based on the total quantity sold).
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,1,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.
Sample Test Cases
Explorer
monthFor...
SUBMIT
Debugger
import pandas as pd
# Prompt the user for the file name
file_name = input()
# Load the data
df = pd.read_csv(file_name)
# write the code..
Average time
0.012 s
12.00 ms
Maximum time
0.015 s
15.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
12 ms
DEBUG
Expected output
sales_data.csv
City sold the most products: Los Angeles
Actual output
sales_data.csv
City sold the most products: Los Angeles
Terminal
Test cases
PREV
RESET
SUBMIT
NEXT


CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.2.4. Most Frequently Sold Product Pairs
00:43
Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:
• The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
• For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
• Output the product pair/s that was sold most frequently.
Sample Data:
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
Explanation:
Transactions:
• 2025-01-01: Product A, Product B
• 2025-01-02: Product A, Product C
• 2025-01-03: Product B, Product A
• 2025-01-04: Product C, Product B
Explorer
frequenti...
SUBMIT
Debugger
import pandas as pd
from itertools import combinations
from collections import Counter
# Prompt user to input the file name
file_name = input()
# Read data from the specified CSV file
df = pd.read_csv(file_name)
Average time
0.010 s
9.67 ms
Maximum time
0.012 s
12.00 ms
1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed
Test case 1
9 ms
DEBUG
Expected output
sales_data.csv
Product A and Product B: 2 times
Product A and Product C: 2 times
Actual output
sales_data.csv
Product A and Product B: 2 times
Product A and Product C: 2 times
Terminal
Test cases
‹ PREV
RESET
SUBMIT
NEXT ›


CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.2.5. Titanic Dataset Analysis and Data Cleaning
00:42
You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.
1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using .info()).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using .describe().
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (mode()).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.
The Titanic dataset contains columns as shown below,
PassengerId
Survived
Pclass
Name
Sex
Age
SibSp
Parch
Ticket
Fare
Cabin
Embarked
Explorer
titanicDat...
SUBMIT
Debugger
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
# 1. Display the first 5 rows
print(data.head())
Average time
0.083 s
83.00 ms
Maximum time
0.083 s
83.00 ms
1 out of 1 shown test case(s) passed
Test case 1
83 ms
DEBUG
Expected output
Actual output
PassengerId Survived Pclass ... Fare Cabin Embarked
0 1 0 3 ... 7.2500 NaN S
1 2 1 1 ... 71.2833 C85 C
2 3 1 3 ... 7.9250 NaN S
Terminal
Test cases
PREV
RESET
SUBMIT
NEXT


CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.2.6. Titanic Dataset Analysis and Data Cleaning - 2
09:02
You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.
1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.
The Titanic dataset contains columns as shown below,
PassengerId
Survived
Pclass
Name
Sex
Age
SibSp
Parch
Ticket
Fare
Cabin
Embarked
Sample Data:
Explorer
titanicDat...
SUBMIT
Debugger
import pandas as pd
import numpy as np
# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')
data['FamilySize'] = data['SibSp'] + data['Parch']
1. IsAlone column
data['IsAlone'] = (data['FamilySize'] == 0).astype(int)
Average time
0.018 s
18.00 ms
Maximum time
0.018 s
18.00 ms
1 out of 1 shown test case(s) passed
Test case 1
18 ms
DEBUG
Expected output
29.69911764705882
14.4542
3 491
1 216
2 184
Name: Pclass, dtype: int64
Actual output
29.69911764705882
14.4542
3 491
1 216
2 184
Name: Pclass, dtype: int64
Terminal
Test cases
PREV
RESET
SUBMIT
NEXT

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

12:08

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

Passengerid	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

Explorer

titanicDat...

SUBMIT

```
1   import pandas as pd
2   import numpy as np
3
4   # Load the Titanic dataset
5   data = pd.read_csv('Titanic-Dataset.csv')
6   data['FamilySize'] = data['SibSp'] + data['Parch']
7   data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8   data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10  # 1. Survival rate by class
```

Debugger

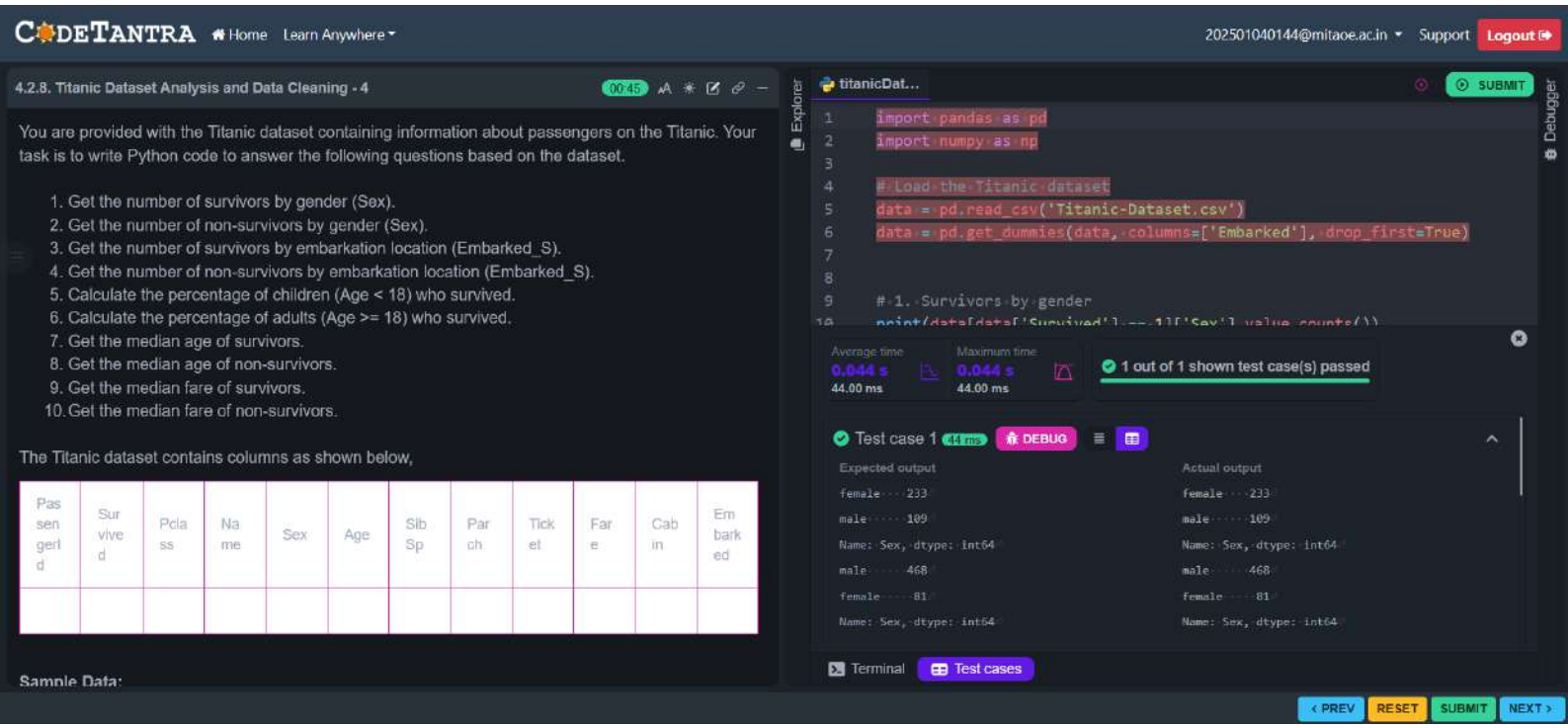
Average time: 0.025 s, 25.00 ms | Maximum time: 0.025 s, 25.00 ms | 1 out of 1 shown test case(s) passed

Test case 1 25 ms DEBUG

Expected output	Actual output
Pclass	Pclass
1 0.629630	1 0.629630
2 0.472826	2 0.472826
3 0.242363	3 0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S	Embarked_S

Terminal Test cases

‹ PREV RESET SUBMIT NEXT ›


CODETANTRA
Home
Learn Anywhere
202501040144@mitaoe.ac.in
Support
Logout
4.2.8. Titanic Dataset Analysis and Data Cleaning - 4
00:45
You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.
1. Get the number of survivors by gender (Sex).
2. Get the number of non-survivors by gender (Sex).
3. Get the number of survivors by embarkation location (Embarked_S).
4. Get the number of non-survivors by embarkation location (Embarked_S).
5. Calculate the percentage of children (Age < 18) who survived.
6. Calculate the percentage of adults (Age >= 18) who survived.
7. Get the median age of survivors.
8. Get the median age of non-survivors.
9. Get the median fare of survivors.
10. Get the median fare of non-survivors.
The Titanic dataset contains columns as shown below,
Passengerid
Survived
Pclass
Name
Sex
Age
SibSp
Parch
Ticket
Fare
Cabin
Embarked
Sample Data:
Explorer
titanicDat...
SUBMIT
Debugger
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9 # 1. Survivors by gender
Average time
0.044 s
44.00 ms
Maximum time
0.044 s
44.00 ms
1 out of 1 shown test case(s) passed
Test case 1
44 ms
DEBUG
Expected output
female 233
male 109
Name: Sex, dtype: int64
male 468
female 81
Name: Sex, dtype: int64
Actual output
female 233
male 109
Name: Sex, dtype: int64
male 468
female 81
Name: Sex, dtype: int64
Terminal
Test cases
< PREV
RESET
SUBMIT
NEXT >

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.1.1. Stacked Plot 39:40

Create a stacked area plot to visualize the temperature variations for three different cities (City A, City B, and City C) across the months of the year. The temperature data is provided for each city in the editor.

Your task is to:

- Create a stacked area plot using the data.
- Label the x-axis as "Month", the y-axis as "Temperature", and provide the title "Temperature Variation" for the plot.
- Display the plot showing the temperature variation for each city throughout the months of the year.

Sample Test Cases +

Explorer

stackedpl... SUBMIT

```
import matplotlib.pyplot as plt
import pandas as pd

# Data for Months and Temperature for three cities
data = {
    'Month': ['January', 'February', 'March', 'April', 'May', 'June',
'July', 'August', 'September', 'October', 'November', 'December'],
    'City_A_Temperature': [5, 7, 10, 13, 17, 20, 22, 21, 18, 12, 8, 6],
    'City_B_Temperature': [2, 3, 5, 6, 10, 14, 16, 17, 12, 9, 5, 3],
```

Debugger

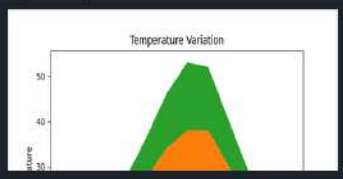
Average time: 0.297 s 297.00 ms

Maximum time: 0.297 s 297.00 ms

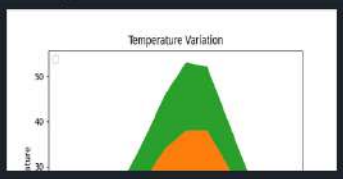
1 out of 1 shown test case(s) passed

Test case 1 297 ms DEBUG

Expected output



Actual output



Terminal Test cases

< PREV RESET SUBMIT NEXT >

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.1. Titanic Dataset 13:21

Write a Python program to analyze and visualize data from the Titanic dataset based on the following instructions:

Dataset Information:

The dataset is stored in a CSV file named `titanic.csv` and has been loaded using the `pandas` library. It contains the following columns:

- `Pclass`: Passenger class (1 = First, 2 = Second, 3 = Third).
- `Gender`: Gender of the passenger (male/female).
- `Age`: Age of the passenger.
- `Survived`: Survival status (0 = Did not survive, 1 = Survived).
- `Fare`: Ticket fare paid by the passenger.

Visualization:

To represent these trends, you will create 5 visualizations using Matplotlib. The visualizations should be arranged in a 3x2 grid (3 rows and 2 columns).

Visualization Details:

Write the code to create a series of visualizations as follows:

Bar Plot (Pclass Distribution):

- Create a bar plot to show the distribution of passengers across the different passenger classes (`Pclass`).

Explorer

titanicDat... SUBMIT

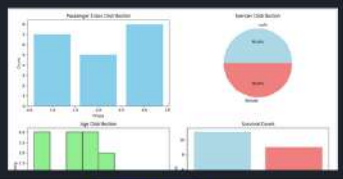
```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset from the CSV file
5  df = pd.read_csv('titanic.csv')
6
7  # Set up the figure for 5 subplots
8  fig, axes = plt.subplots(3, 2, figsize=(12, 12))
9
10 # 1. Passenger Class Distribution
```

Debugger

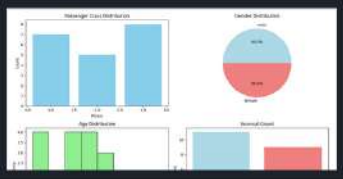
Average time 0.803 s 803.00 ms | Maximum time 0.803 s 803.00 ms | 1 out of 1 shown test case(s) passed

Test case 1 803 ms DEBUG

Expected output



Actual output



Terminal Test cases

‹ PREV RESET SUBMIT NEXT ›

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.2. Histogram of passenger information of Titanic

01:48

Write a Python code to plot a histogram for the distribution of the 'Age' column from the Titanic dataset. The histogram should display the frequency of different age ranges with the following specifications:

1. Use **30 bins** for the histogram.
2. Set the **edge color** of the bars to **black** (k).
3. Label the x-axis as **'Age'** and the y-axis as **'Frequency'**.
4. Add the title **"Age Distribution"** to the histogram.

The Titanic dataset contains columns as shown below,

Passengerid	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
```

Explorer

Histogra...

SUBMIT

Debugger

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
```

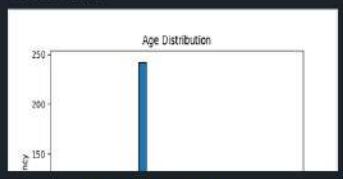
Average time: 0.208 s, 208.00 ms

Maximum time: 0.208 s, 208.00 ms

1 out of 1 shown test case(s) passed

Test case 1 208 ms DEBUG

Expected output



Actual output

Terminal Test cases

‹ PREV RESET SUBMIT NEXT ›

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.3. Bar plot of survival rate of passengers

06:28

Write a Python code to plot a bar chart that shows the count of passengers who survived and did not survive in the Titanic dataset. The chart should display the following specifications:

1. Use the '**Survived**' column to show the count of survivors (0 = Did not survive, 1 = Survived).
2. Set the chart type to '**bar**'.
3. Add the title "**Survival Count**" to the chart.
4. Label the x-axis as '**Survived**' and the y-axis as '**Count**'.

The Titanic dataset contains columns as shown below,

Passengerid	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
```

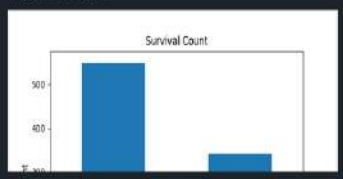
BarPlotOf... SUBMIT

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
```

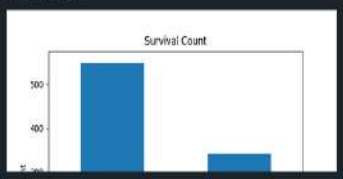
Average time 0.257 s 267.00 ms | Maximum time 0.257 s 267.00 ms | 1 out of 1 shown test case(s) passed

Test case 1 267 ms DEBUG

Expected output



Actual output



Terminal Test cases

< PREV RESET SUBMIT NEXT >

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.4. Bar Plot for Survival by Gender

25:38

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by gender, in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the **'Sex'** column, then use the **value_counts()** function to count the occurrences of survivors (0 = Did not survive, 1 = Survived) for each gender.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title **"Survival by Gender"** to the chart.
4. Label the x-axis as **'Gender'** and the y-axis as **'Count'**.
5. The legend should indicate **'Not Survived'** and **'Survived'**.

The Titanic dataset contains columns as shown below,

Passengerid	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
```

Explorer

BarPlotOf...

SUBMIT

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

Debugger

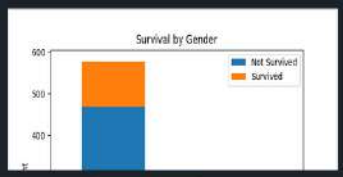
Average time 0.266 s 266.00 ms

Maximum time 0.266 s 266.00 ms

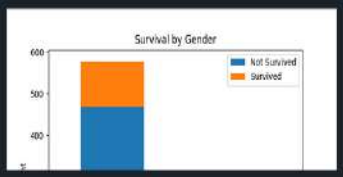
1 out of 1 shown test case(s) passed

Test case 1 266 ms DEBUG

Expected output



Actual output



Terminal Test cases

‹ PREV RESET SUBMIT NEXT ›

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.5. Bar Plot for Survival by Pclass

05:58

Write a Python code to plot a stacked bar chart that shows the count of passengers who survived and did not survive, grouped by passenger class (**Pclass**), in the Titanic dataset. The chart should display the following specifications:

1. Group the data by the **Pclass** column and count the number of survivors (0 = Did not survive, 1 = Survived) for each class using **value_counts()**.
2. Use a **stacked bar chart** to display the survival counts.
3. Add the title **"Survival by Pclass"** to the chart.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Count'**.
5. The legend should indicate **'Not Survived'** and **'Survived'**.

The Titanic dataset contains columns as shown below,

Passengerid	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
```

Explorer

BarPlotOf...

SUBMIT

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
```

Debugger

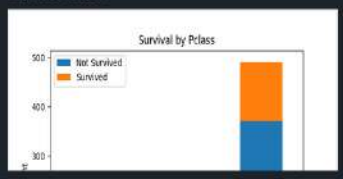
Average time: 0.271 s, 271.00 ms

Maximum time: 0.271 s, 271.00 ms

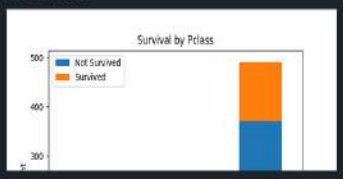
1 out of 1 shown test case(s) passed

Test case 1 271 ms DEBUG

Expected output



Actual output



Terminal Test cases

‹ PREV RESET SUBMIT NEXT ›

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.6. Bar Plot for Survival by Embarked

09:21

Write a Python code to plot a stacked bar chart showing the survival count for passengers based on their embarkation location in the Titanic dataset.
The chart should display the following specifications:

1. Use the **Embarked** column to determine the embarkation location. After converting this column into dummy variables (using **pd.get_dummies()**), plot the survival count based on the **Embarked_Q** column (representing passengers who embarked from Queenstown) in relation to survival.
2. Set the chart type to 'bar' and make it stacked.
3. Add the title **"Survival by Embarked "** to the chart.
4. Label the x-axis as **'Embarked'** and the y-axis as **'Count'**.
5. Include a legend to distinguish between survivors and non-survivors (label the legend as **'Survived'** and **'Not Survived'**).

The Titanic dataset contains columns as shown below,

Passengerid	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

Explorer

BarPlotOf...

SUBMIT

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
```

Debugger

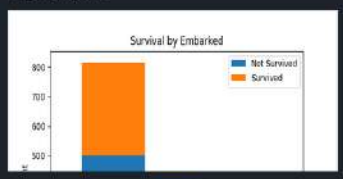
Average time 0.276 s 276.00 ms

Maximum time 0.276 s 276.00 ms

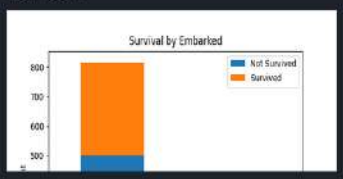
1 out of 1 shown test case(s) passed

Test case 1 276 ms DEBUG

Expected output



Actual output



Terminal Test cases

< PREV RESET SUBMIT NEXT >

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.7. Box plot for Age Distribution

10:01

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset across different passenger classes. The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Age by Pclass"**.
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below,

Passengerld	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
```

Explorer

BoxPlotF...

SUBMIT

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
```

Debugger

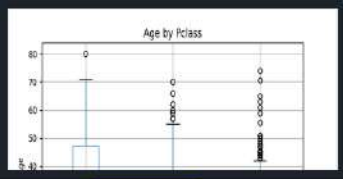
Average time 0.283 s 283.00 ms

Maximum time 0.283 s 283.00 ms

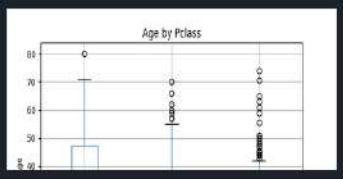
1 out of 1 shown test case(s) passed

Test case 1 283 ms DEBUG

Expected output



Actual output



Terminal Test cases

‹ PREV RESET SUBMIT NEXT ›

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.8. Box Plot for Age by Survived

03:19

Write a Python code to plot a boxplot that shows the distribution of the 'Age' column from the Titanic dataset based on whether passengers survived or not. The boxplot should display the following specifications:

1. Use the **Survived** column to group the data for the boxplot (0 = Did not survive, 1 = Survived).
2. Set the title of the plot to **"Age by Survival"**.
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as **'Survived'** and the y-axis as **'Age'**.

The Titanic dataset contains columns as shown below,

Passengerid	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
```

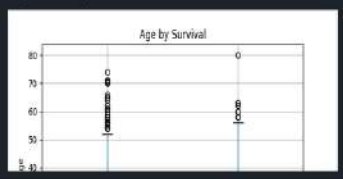
BoxPlotF... SUBMIT

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
```

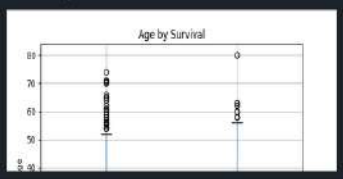
Average time: 0.289 s, 289.00 ms | Maximum time: 0.289 s, 289.00 ms | 1 out of 1 shown test case(s) passed

Test case 1 289 ms DEBUG

Expected output



Actual output



Terminal Test cases

‹ PREV RESET SUBMIT NEXT ›

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.9. Box Plot for Fare by Pclass

02:16

Write a Python code to plot a boxplot that shows the distribution of the 'Fare' column from the Titanic dataset based on the passenger class (Pclass). The boxplot should display the following specifications:

1. Use the **Pclass** column to group the data for the boxplot.
2. Set the title of the plot to **"Fare by Pclass"**.
3. Remove the default subtitle with **plt.suptitle('')**.
4. Label the x-axis as **'Pclass'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below,

Passengerld	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
```

Explorer

BoxPlotF...

SUBMIT

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

Debugger

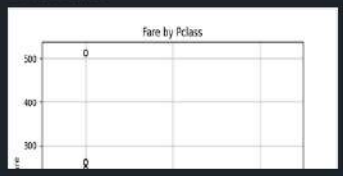
Average time 0.261 s 261.00 ms

Maximum time 0.261 s 261.00 ms

1 out of 1 shown test case(s) passed

Test case 1 261 ms DEBUG

Expected output



Actual output

Terminal Test cases

< PREV RESET SUBMIT NEXT >

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.10. Scatter Plot for Age vs. Fare

03:16

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Set the title of the plot to **"Age vs. Fare"**.
3. Label the x-axis as **'Age'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below,

Passengerid	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
```

Explorer

AgeFareS...

SUBMIT

```
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  # Load the Titanic dataset
5  data = pd.read_csv('Titanic-Dataset.csv')
6
7  # Data Cleaning
8  data['Age'].fillna(data['Age'].median(), inplace=True)
9  data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
10 data.drop('Cabin', axis=1, inplace=True)
```

Debugger

Average time: 0.180 s, 180.00 ms

Maximum time: 0.180 s, 180.00 ms

1 out of 1 shown test case(s) passed

Test case 1 180 ms DEBUG

Expected output

Actual output

Terminal Test cases

‹ PREV RESET SUBMIT NEXT ›

CODETANTRA Home Learn Anywhere 202501040144@mitaoe.ac.in Support Logout

5.2.11. Scatter Plot for Age vs. Fare by Survived

20:12

Write a Python code to plot a scatter plot showing the relationship between the 'Age' and 'Fare' columns in the Titanic dataset, with points color-coded by survival status. The scatter plot should display the following specifications:

1. Use the **Age** column for the x-axis and the **Fare** column for the y-axis.
2. Color the points based on the **Survived** column: **Red** for passengers who did not survive (**Survived = 0**). **Blue** for passengers who survived (**Survived = 1**).
3. Set the title of the plot to **"Age vs. Fare by Survival"**.
4. Label the x-axis as **'Age'** and the y-axis as **'Fare'**.

The Titanic dataset contains columns as shown below,

Passengerid	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

```
PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked
1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S
2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC 17599,71.2833,C85,C
3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
```

AgeFareS...

SUBMIT

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the Titanic dataset
data = pd.read_csv('Titanic-Dataset.csv')

# Data Cleaning
data['Age'].fillna(data['Age'].median(), inplace=True)
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

Average time: 0.212 s, 212.00 ms

Maximum time: 0.212 s, 212.00 ms

1 out of 1 shown test case(s) passed

Test case 1 212 ms DEBUG

Expected output

Actual output

Terminal Test cases

PREV RESET SUBMIT